


```

#!/usr/bin/env python3
import math
import numpy as np

try:
    import scipy.sparse as sp
    import scipy.linalg as la
except ImportError as e:
    raise SystemExit("Need scipy (pip install scipy).") from e

def idx3(x, y, z, L):
    return x + L * (y + L * z)

def mod(x, L):
    return x % L

def build_ops_3d_torus(L: int):
    """
    3D periodic cubic lattice ( $T^3$ ).
    0-cells: vertices  $v=(x,y,z)$ 
    1-cells: oriented edges  $e=(v,\mu)$ ,  $\mu=0,1,2$  (positive direction)
    2-cells: oriented faces  $f=(v,(\mu,\nu))$  with  $(01),(02),(12)$ 
    3-cells: cubes  $c=(v)$ 
    """
    n0 = L**3
    n1 = 3 * L**3
    n2 = 3 * L**3
    n3 = L**3

    pairs = [(0,1),(0,2),(1,2)]
    pair_to_id = {pairs[i]: i for i in range(3)}
    dirs = [(1,0,0),(0,1,0),(0,0,1)]

    def v_id(x,y,z): return idx3(x,y,z,L)
    def e_id(v, mu): return 3*v + mu
    def f_id(v, mu, nu):
        if mu > nu: mu, nu = nu, mu
        return 3*v + pair_to_id[(mu,nu)]
    def c_id(v): return v

    # d0:  $V \rightarrow E$ ,  $(d_0 \phi)(v,\mu) = \phi(v+\mu) - \phi(v)$ 
    rows, cols, data = [], [], []
    for z in range(L):
        for y in range(L):
            for x in range(L):
                v = v_id(x,y,z)
                for mu, (dx,dy,dz) in enumerate(dirs):
                    vp = v_id(mod(x+dx,L), mod(y+dy,L), mod(z+dz,L))
                    r = e_id(v, mu)

```

```

        rows += [r, r]
        cols += [vp, v]
        data += [1.0, -1.0]
d0 = sp.csr_matrix((data,(rows,cols)), shape=(n1,n0))

# d1: E->F, (d1 X)(face v,mu,nu) = X(v,mu) + X(v+mu,nu) - X(v+nu,mu) - X
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            for (mu,nu) in pairs:
                r = f_id(v, mu, nu)
                dxm,dym,dzm = dirs[mu]
                dxn,dyn,dzn = dirs[nu]
                v_mu = v_id(mod(x+dxm,L), mod(y+dym,L), mod(z+dzm,L))
                v_nu = v_id(mod(x+dxn,L), mod(y+dyn,L), mod(z+dzn,L))
                rows.append(r); cols.append(e_id(v,mu)); data.append(
                rows.append(r); cols.append(e_id(v_mu,nu)); data.append(
                rows.append(r); cols.append(e_id(v_nu,mu)); data.append(
                rows.append(r); cols.append(e_id(v,nu)); data.append(
d1 = sp.csr_matrix((data,(rows,cols)), shape=(n2,n1))

# d2: F->C (discrete Bianchi) chosen so that d2 d1 = 0 with our orientat
# (d2 F)(cube v) =
# +F12(v+e0) - F12(v)
# -F02(v+e1) + F02(v)
# +F01(v+e2) - F01(v)
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            r = c_id(v)
            v_e0 = v_id(mod(x+1,L), y, z)
            v_e1 = v_id(x, mod(y+1,L), z)
            v_e2 = v_id(x, y, mod(z+1,L))

            rows += [r, r]; cols += [f_id(v_e0,1,2), f_id(v,1,2)]; data
            rows += [r, r]; cols += [f_id(v_e1,0,2), f_id(v,0,2)]; data
            rows += [r, r]; cols += [f_id(v_e2,0,1), f_id(v,0,1)]; data
d2 = sp.csr_matrix((data,(rows,cols)), shape=(n3,n2))

return d0, d1, d2, (n0,n1,n2,n3)

def analyze(L, m2=0.3, alpha=1.0, tol_zero=1e-9):
    d0, d1, d2, (n0,n1,n2,n3) = build_ops_3d_torus(L)

    # Bianchi: d2 d1 = 0
    B = (d2 @ d1).tocoo()
    bianchi norm = float(nn.sqrt((B.data**2).sum())) if B.nnz else 0.0

```

```

# Ranks (dense for modest L)
D0 = d0.toarray()
D1 = d1.toarray()
r0 = int(np.linalg.matrix_rank(D0))
r1 = int(np.linalg.matrix_rank(D1))

# b1 = dim ker(d1) - dim im(d0)
dim_ker_d1 = n1 - r1
b1 = dim_ker_d1 - r0

# Vacuum Hessian (per Lie-algebra component): A = alpha d1^T d1
A = alpha * (d1.T @ d1).toarray()

# Coulomb gauge subspace: d0^T x = 0
N = la.null_space(D0.T) # orthonormal basis columns
A_sub = N.T @ (A @ N)
evals = np.linalg.eigvalsh(A_sub)
evals.sort()

n_zero = int(np.sum(evals <= tol_zero))
pos = evals[evals > tol_zero]
lam_min_pos = float(pos[0]) if pos.size else float("nan")

# Massive: m^2 I + A
evals_M = np.linalg.eigvalsh((m2*np.eye(A_sub.shape[0])) + A_sub)
evals_M.sort()

lap_scale = 4.0 * (math.sin(math.pi / L)**2)

print(f"\n=== L={L} on T^3 ===")
print(f"dims: n0={n0}, n1={n1}, n2={n2}, n3={n3}")
print(f"||d2 d1||_F = {bianchi_norm:.3e} (Bianchi: should be ~0)")
print(f"rank(d0)={r0}, rank(d1)={r1}, b1={b1} (expect 3 on T^3)")
print(f"Coulomb gauge: zero multiplicity = {n_zero} (expect 3 harmonic)")
print(f"min positive eig(d1^T d1 | Coulomb) = {lam_min_pos:.6g}")
print(f"4 sin^2(pi/L) = {lap_scale:.6g} | ratio = {lam_min_pos/lap_scale}")
print(f"With mass m^2={m2}: min eig(Coulomb) = {evals_M[0]:.6g} (expect

def main():
    for L in [4,5,6]:
        analyze(L, m2=0.3, alpha=1.0)

if __name__ == "__main__":
    main()

```

```

=== L=4 on T^3 ===
dims: n0=64, n1=192, n2=192, n3=64
||d2 d1||_F = 0.00000000 (Bianchi: should be ~0)

```

```

||d2 d1||_F = 0.000000 (Bianchi: should be ~0)
rank(d0)=63, rank(d1)=126, b1=3 (expect 3 on T^3)
Coulomb gauge: zero multiplicity = 3 (expect 3 harmonic)
min positive eig(d1^T d1 | Coulomb) = 2
4 sin^2(pi/L) = 2 | ratio = 1
With mass m^2=0.3: min eig(Coulomb) = 0.3 (expect ~m^2)

=== L=5 on T^3 ===
dims: n0=125, n1=375, n2=375, n3=125
||d2 d1||_F = 0.000e+00 (Bianchi: should be ~0)
rank(d0)=124, rank(d1)=248, b1=3 (expect 3 on T^3)
Coulomb gauge: zero multiplicity = 3 (expect 3 harmonic)
min positive eig(d1^T d1 | Coulomb) = 1.38197
4 sin^2(pi/L) = 1.38197 | ratio = 1
With mass m^2=0.3: min eig(Coulomb) = 0.3 (expect ~m^2)

=== L=6 on T^3 ===
dims: n0=216, n1=648, n2=648, n3=216
||d2 d1||_F = 0.000e+00 (Bianchi: should be ~0)
rank(d0)=215, rank(d1)=430, b1=3 (expect 3 on T^3)
Coulomb gauge: zero multiplicity = 3 (expect 3 harmonic)
min positive eig(d1^T d1 | Coulomb) = 1
4 sin^2(pi/L) = 1 | ratio = 1
With mass m^2=0.3: min eig(Coulomb) = 0.3 (expect ~m^2)

```

```

import numpy as np

L = 6
d0, d1, d2, (n0, n1, n2, n3) = build_ops_3d_torus(L)

phi = np.random.randn(n0)
x = d0 @ phi
v = d1 @ x
print("||d1 d0 phi|| =", np.linalg.norm(np.asarray(v).ravel()))

y = np.random.randn(n3)
s = d2.T @ y
w = d1.T @ s
print("||d1^T d2^T y|| =", np.linalg.norm(np.asarray(w).ravel()))

```

```

||d1 d0 phi|| = 4.355884775951763e-15
||d1^T d2^T y|| = 4.159119551633463e-15

```

```

#!/usr/bin/env python3
import math
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

# -----
# Lattice cochain operators on T^3
..

```

```

# -----

def idx3(x, y, z, L):
    return x + L * (y + L * z)

def inv_idx3(v, L):
    x = v % L
    v //= L
    y = v % L
    z = v // L
    return x, y, z

def mod(x, L):
    return x % L

def build_ops_3d_torus(L: int):
    """
    0-cells: vertices v=(x,y,z)
    1-cells: edges e=(v,mu), mu=0,1,2
    2-cells: faces f=(v, (01),(02),(12))
    3-cells: cubes c=(v)
    """
    n0 = L**3
    n1 = 3 * L**3
    n2 = 3 * L**3
    n3 = L**3

    pairs = [(0,1),(0,2),(1,2)]
    pair_to_id = {pairs[i]: i for i in range(3)}
    dirs = [(1,0,0),(0,1,0),(0,0,1)]

    def v_id(x,y,z): return idx3(x,y,z,L)
    def e_id(v, mu): return 3*v + mu
    def f_id(v, mu, nu):
        if mu > nu: mu, nu = nu, mu
        return 3*v + pair_to_id[(mu,nu)]
    def c_id(v): return v

    # d0: V -> E, (d0 phi)(v,mu)=phi(v+mu)-phi(v)
    rows, cols, data = [], [], []
    for z in range(L):
        for y in range(L):
            for x in range(L):
                v = v_id(x,y,z)
                for mu, (dx,dy,dz) in enumerate(dirs):
                    vp = v_id(mod(x+dx,L), mod(y+dy,L), mod(z+dz,L))
                    r = e_id(v, mu)
                    rows += [r, r]
                    cols += [vp, v]
                    data += [1.0, -1.0]
    d0 = sp.csr_matrix((data,(rows,cols)), shape=(n1,n0))

```

```

# d1: E -> F, (d1 X)(v,mu,nu) = X(v,mu) + X(v+mu,nu) - X(v+nu,mu) - X(v,nu)
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            for (mu,nu) in pairs:
                r = f_id(v, mu, nu)
                dxm,dym,dzm = dirs[mu]
                dxn,dyn,dzn = dirs[nu]
                v_mu = v_id(mod(x+dxm,L), mod(y+dym,L), mod(z+dzm,L))
                v_nu = v_id(mod(x+dxn,L), mod(y+dyn,L), mod(z+dzn,L))
                rows.append(r); cols.append(e_id(v,mu)); data.append(1)
                rows.append(r); cols.append(e_id(v_mu,nu)); data.append(1)
                rows.append(r); cols.append(e_id(v_nu,mu)); data.append(-1)
                rows.append(r); cols.append(e_id(v,nu)); data.append(-1)
d1 = sp.csr_matrix((data,(rows,cols)), shape=(n2,n1))

# d2: F -> C (Bianchi), chosen so that d2 d1 = 0 with conventions above
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            r = c_id(v)
            v_e0 = v_id(mod(x+1,L), y, z)
            v_e1 = v_id(x, mod(y+1,L), z)
            v_e2 = v_id(x, y, mod(z+1,L))
            rows += [r, r]; cols += [f_id(v_e0,1,2), f_id(v,1,2)]; data += [1, -1]
            rows += [r, r]; cols += [f_id(v_e1,0,2), f_id(v,0,2)]; data += [1, -1]
            rows += [r, r]; cols += [f_id(v_e2,0,1), f_id(v,0,1)]; data += [1, -1]
d2 = sp.csr_matrix((data,(rows,cols)), shape=(n3,n2))

return d0, d1, d2, (n0,n1,n2,n3)

# -----
# Distances between edges (L1 on edge-centers with torus wrap)
# -----

def edge_center_halfcoords(e, L):
    """
    Edge index e = 3*v + mu.
    Return center coordinates on half-lattice (integers mod 2L):
    mu=0: (2x+1, 2y, 2z)
    mu=1: (2x, 2y+1, 2z)
    mu=2: (2x, 2y, 2z+1)
    """
    v, mu = divmod(int(e), 3)
    x,y,z = inv_idx3(v, L)
    if mu == 0:
        return (2*x+1, 2*y, 2*z)
    elif mu == 1:
        return (2*x, 2*y+1, 2*z)
    else:
        return (2*x, 2*y, 2*z+1)

```

```

    if mu == 0: return (2*x+1, 2*y, 2*z)
    if mu == 1: return (2*x, 2*y+1, 2*z)
    return (2*x, 2*y, 2*z+1)

def torus_L1_halfdist(a, b, L):
    """
    L1 distance on half-lattice with period 2L in each coord.
    Returns integer r_half = dx+dy+dz (in half-units).
    """
    P = 2*L
    r = 0
    for i in range(3):
        da = (a[i] - b[i]) % P
        da = min(da, P - da)
        r += da
    return int(r)

# -----
# Green's function decay experiment
# -----

def solve_column(M, src_idx, tol=1e-12, maxiter=5000):
    """
    Solve  $M g = e_{\text{src}}$ . Prefer CG; fall back to sparse direct if needed.
    """
    n = M.shape[0]
    b = np.zeros(n)
    b[src_idx] = 1.0

    # CG (SPD)
    g, info = spla.cg(M, b, rtol=tol, atol=0.0, maxiter=maxiter)
    if info == 0:
        return g

    # fallback: sparse direct
    return spla.spsolve(M, b)

def envelope_by_distance(g, L, src_edge):
    """
    Compute envelope( $r$ ) = max_{edges at distance  $r$ } |g_edge|
    using  $r$  in half-units, then report  $r = r_{\text{half}}/2$ .
    """
    src_c = edge_center_halfcoords(src_edge, L)
    env = {} # r_half -> maxabs

    for e in range(g.shape[0]):
        c = edge_center_halfcoords(e, L)
        r_half = torus_L1_halfdist(src_c, c, L)
        val = abs(float(g[e]))
        if r_half not in env or val > env[r_half]:
            env[r_half] = val

```

```

r_half_sorted = np.array(sorted(env.keys()), dtype=int)
r = r_half_sorted / 2.0
y = np.array([env[k] for k in r_half_sorted], dtype=float)
return r, y

def fit_decay(r, env, r_min=2.0, r_max=None, eps=1e-300):
    """
    Fit log(env) = a - c r on a chosen range.
    """
    if r_max is None:
        r_max = float(np.max(r))
    mask = (r >= r_min) & (r <= r_max) & (env > 0)
    rr = r[mask]
    yy = np.log(np.maximum(env[mask], eps))
    if rr.size < 2:
        return float("nan"), float("nan"), int(rr.size)
    # least squares slope
    A = np.vstack([np.ones_like(rr), rr]).T
    a, slope = np.linalg.lstsq(A, yy, rcond=None)[0]
    c = -float(slope)
    return float(a), float(c), int(rr.size)

def main():
    # Tunables
    L = 10          # keep modest; n1 = 3 L^3
    alpha = 1.0
    m2 = 0.30       # mass^2 in M = m^2 I + alpha d1^T d1
    src_edge = 0    # edge at v=0, mu=0

    d0, d1, d2, (n0,n1,n2,n3) = build_ops_3d_torus(L)

    # Build M on edges
    A = (d1.T @ d1).tocsr()
    M = (m2 * sp.eye(n1, format="csr")) + (alpha * A)

    # Solve one Green's function column
    g = solve_column(M, src_edge)

    # Envelope vs distance
    r, env = envelope_by_distance(g, L, src_edge)

    # Fit decay rate
    a_fit, c_fit, nfit = fit_decay(r, env, r_min=2.0, r_max=min(np.max(r), L/2))

    m = math.sqrt(m2)
    c_pred = math.asinh(m / alpha)    # the heuristic you requested

    print(f"=== Massive edge Green's function on T^3 ===")
    print(f"L={L}, n1={n1}, alpha={alpha}, m^2={m2} (m={m:.6g})")
    print(f"Fit: a_fit={a_fit}, c_fit={c_fit}, nfit={nfit} vs heuristic c_pred={c_pred}")

```

```

print(f"Fit range: r in [{2.0}, {min(np.max(r), L/2):.6g}] with {nfit} bins")
print(f"c_fit = {c_fit:.6g}")
print(f"c_pred = asinh(m/alpha) = {c_pred:.6g}")
print()

# Print a small table of envelope values
print("r    envelope_max|g|    log(envelope)")
for rr, ee in list(zip(r, env))[:25]:
    print(f"{rr:4.1f}  {ee: .3e}          {math.log(max(ee,1e-300)):+.3f}")

# And a few far points
print("\n... tail samples ...")
for rr, ee in list(zip(r, env))[-10:]:
    print(f"{rr:4.1f}  {ee: .3e}          {math.log(max(ee,1e-300)):+.3f}")

if __name__ == "__main__":
    main()

```

```

=== Massive edge Green's function on T^3 ===
L=10, n1=3000, alpha=1.0, m^2=0.3 (m=0.547723)
Fit range: r in [2.0, 5] with 4 bins
c_fit = 0.825908
c_pred = asinh(m/alpha) = 0.523484

```

r	envelope_max g	log(envelope)
0.0	1.249e+00	+0.223
1.0	4.328e-01	-0.838
2.0	8.451e-02	-2.471
3.0	3.036e-02	-3.495
4.0	1.436e-02	-4.243
5.0	6.913e-03	-4.974
6.0	4.156e-03	-5.483
7.0	2.736e-03	-5.901
8.0	2.139e-03	-6.147
9.0	1.621e-03	-6.425
10.0	1.272e-03	-6.667
11.0	1.151e-03	-6.767
12.0	9.067e-04	-7.006
13.0	6.540e-04	-7.332
14.0	4.796e-04	-7.643
15.0	4.188e-04	-7.778

... tail samples ...

6.0	4.156e-03	-5.483
7.0	2.736e-03	-5.901
8.0	2.139e-03	-6.147
9.0	1.621e-03	-6.425
10.0	1.272e-03	-6.667
11.0	1.151e-03	-6.767
12.0	9.067e-04	-7.006
13.0	6.540e-04	-7.332
14.0	4.796e-04	-7.643
15.0	4.188e-04	-7.778

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# -----
# Symbols and Green's function on  $T^D$ 
# -----

def lam_symbol_Td(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k) # [L]
    lam = t
    for _ in range(D-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, D):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(D-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=False):
    lam = lam_symbol_Td(L, D, device=device)
    denom = m2 + alpha * (lam ** R_power)
    if remove_k0:
        denom[(0,)*D] = float("inf")
    Gk = 1.0 / denom
    g = torch.fft.ifftn(Gk).real
    return g

def torus_L1_dist_grid(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(D-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, D):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(D-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

```

```

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=torch.float64)
    return mids, slopes

def fit_stats(mids, slopes, rmin, rmax):
    mask = (mids >= rmin) & (mids <= rmax) & torch.isfinite(slopes)
    return (
        float(torch.median(slopes[mask]).item()),
        float(torch.mean(slopes[mask]).item()),
        int(mask.sum().item())
    )

# -----
# Directional decay diagnostics (fixed)
# -----

def directional_profile_abs(g, direction, L, D, rmax):
    direction = tuple(int(x) for x in direction)
    vals = torch.zeros(rmax+1, device=g.device, dtype=torch.float64)
    for r in range(rmax+1):
        idx = tuple((r*direction[i]) % L for i in range(D))
        vals[r] = torch.abs(g[idx])
    return vals

def tail_envelope(vals):
    # tail_env[r] = max_{s>=r} vals[s], monotone decreasing
    out = vals.clone()
    for r in range(out.numel()-2, -1, -1):
        out[r] = torch.maximum(out[r], out[r+1])
    return out

def directional_slope_from_tail(vals, rmin, rmax):
    tail = tail_envelope(vals)
    y = torch.log(torch.clamp(tail, min=1e-300))
    slopes = -(y[1:] - y[:-1]) # Δr=1
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=vals.device, dtype=torch.float64)
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.median(slopes[mask]).item()), float(torch.mean(slopes[r
# -----

```

```

# Run (d=4) for R=1 only (tighten the manuscript narrative)
# -----

D = 4
L = 64          # bigger L => more safe mid-range; A100 can handle 64^4
m2 = 0.3
alpha = 1.0
R_power = 1

# Safe windows: keep well below wrap. L/2 = 32.
rmin_fit = 6.0
rmax_fit_shell = 20.0
rmax_dir = 24    # still < 32, avoids wrap artifacts along a line

g = greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=False) # no need to drop
abs_g = torch.abs(g)

# L1-shell envelope
dist = torus_L1_dist_grid(L, D, device=device)
r_max_total = D * (L//2)
env = env_by_dist(abs_g, dist, r_max_total)
mids, slopes = local_slopes(env)
c_med, c_mean, npts = fit_stats(mids, slopes, rmin_fit, rmax_fit_shell)

# Directional tail-envelope slopes (monotone by construction)
axis = (1,0,0,0)
diag2 = (1,1,0,0)
diag4 = (1,1,1,1)

v_axis = directional_profile_abs(g, axis, L, D, rmax_dir)
v_d2 = directional_profile_abs(g, diag2, L, D, rmax_dir)
v_d4 = directional_profile_abs(g, diag4, L, D, rmax_dir)

s_axis_med, s_axis_mean = directional_slope_from_tail(v_axis, rmin_fit, rmax_dir)
s_d2_med, s_d2_mean = directional_slope_from_tail(v_d2, rmin_fit, rmax_dir)
s_d4_med, s_d4_mean = directional_slope_from_tail(v_d4, rmin_fit, rmax_dir)

kappa_axis = math.acosh(1.0 + m2/(2.0*alpha)) # exact axis benchmark on Z (directional)

print("=== FFT (exact) decay diagnostics for M = m^2 I + alpha Δ on T^4 ===")
print(f"device={device}, d={D}, L={L}, m^2={m2}, alpha={alpha}, m={math.sqrt(m2)}")
print(f"L1-shell fit mids in [{rmin_fit}, {rmax_fit_shell}] (bins={npts})")
print(f" c_shell median = {c_med:.6g}")
print(f" c_shell mean = {c_mean:.6g}")
print()
print(f"Directional (tail-envelope) median slopes, r in [{rmin_fit}, {rmax_dir}]")
print(f" axis (1,0,0,0): {s_axis_med:.6g} (mean {s_axis_mean:.6g})")
print(f" diag2 (1,1,0,0): {s_d2_med:.6g} (mean {s_d2_mean:.6g})")
print(f" diag4 (1,1,1,1): {s_d4_med:.6g} (mean {s_d4_mean:.6g})")
print()

```

```

print(f"kappa_axis benchmark = acosh(1 + m^2/(2alpha)) = {kappa_axis:.6g}")
print()

# Print a short shell table
for r in range(0, 26):
    val = float(env[r].item())
    logv = math.log(max(val, 1e-300))
    print(f"r={r:2d}  env={val:.3e}  log={logv:+.3f}")

```

```

=== FFT (exact) decay diagnostics for  $M = m^2 I + \alpha \Delta$  on  $T^4$  ===
device=cuda, d=4, L=64,  $m^2=0.3$ ,  $\alpha=1.0$ ,  $m=0.547723$ 
L1-shell fit mids in [6.0, 20.0] (bins=14)
  c_shell median = 0.390352
  c_shell mean   = 0.395472

```

```

Directional (tail-envelope) median slopes, r in [6.0, 23]
axis  (1,0,0,0):  0.64957   (mean 0.668088)
diag2 (1,1,0,0):  0.875545   (mean 0.891714)
diag4 (1,1,1,1):  1.19668    (mean 1.21183)

```

```

kappa_axis benchmark = acosh(1 + m^2/(2alpha)) = 0.541097

```

```

r= 0  env=1.439e-01  log=-1.939
r= 1  env=2.426e-02  log=-3.719
r= 2  env=8.724e-03  log=-4.742
r= 3  env=4.533e-03  log=-5.396
r= 4  env=2.859e-03  log=-5.857
r= 5  env=1.400e-03  log=-6.572
r= 6  env=7.976e-04  log=-7.134
r= 7  env=5.032e-04  log=-7.594
r= 8  env=3.406e-04  log=-7.985
r= 9  env=2.035e-04  log=-8.500
r=10  env=1.301e-04  log=-8.947
r=11  env=8.754e-05  log=-9.343
r=12  env=6.127e-05  log=-9.700
r=13  env=3.960e-05  log=-10.137
r=14  env=2.664e-05  log=-10.533
r=15  env=1.850e-05  log=-10.898
r=16  env=1.320e-05  log=-11.235
r=17  env=8.886e-06  log=-11.631
r=18  env=6.147e-06  log=-11.999
r=19  env=4.352e-06  log=-12.345
r=20  env=3.142e-06  log=-12.671
r=21  env=2.168e-06  log=-13.041
r=22  env=1.527e-06  log=-13.392
r=23  env=1.095e-06  log=-13.725
r=24  env=7.971e-07  log=-14.042
r=25  env=5.592e-07  log=-14.397

```

```

#!/usr/bin/env python3
import math
import numpy as np
import scipy.sparse as sp

```

```

import scipy.sparse as sp
import scipy.sparse.linalg as spla

# -----
# Lattice cochain operators on  $T^3$ 
# -----

def idx3(x, y, z, L):
    return x + L * (y + L * z)

def inv_idx3(v, L):
    x = v % L
    v //= L
    y = v % L
    z = v // L
    return x, y, z

def mod(x, L):
    return x % L

def build_ops_3d_torus(L: int):
    """
    0-cells: vertices v=(x,y,z)
    1-cells: edges e=(v,mu), mu=0,1,2
    2-cells: faces f=(v, (01),(02),(12))
    3-cells: cubes c=(v)
    """
    n0 = L**3
    n1 = 3 * L**3
    n2 = 3 * L**3
    n3 = L**3

    pairs = [(0,1),(0,2),(1,2)]
    pair_to_id = {pairs[i]: i for i in range(3)}
    dirs = [(1,0,0),(0,1,0),(0,0,1)]

    def v_id(x,y,z): return idx3(x,y,z,L)
    def e_id(v, mu): return 3*v + mu
    def f_id(v, mu, nu):
        if mu > nu: mu, nu = nu, mu
        return 3*v + pair_to_id[(mu,nu)]
    def c_id(v): return v

    # d0: V -> E, (d0 phi)(v,mu)=phi(v+mu)-phi(v)
    rows, cols, data = [], [], []
    for z in range(L):
        for y in range(L):
            for x in range(L):
                v = v_id(x,y,z)
                for mu, (dx,dy,dz) in enumerate(dirs):
                    vp = v_id(mod(x+dx,L), mod(y+dy,L), mod(z+dz,L))

```

```

        r = e_id(v, mu)
        rows += [r, r]
        cols += [vp, v]
        data += [1.0, -1.0]
d0 = sp.csr_matrix((data,(rows,cols)), shape=(n1,n0))

# d1: E -> F, (d1 X)(v,mu,nu) = X(v,mu) + X(v+mu,nu) - X(v+nu,mu) - X(v,
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            for (mu,nu) in pairs:
                r = f_id(v, mu, nu)
                dxm,dym,dzm = dirs[mu]
                dxn,dyn,dzn = dirs[nu]
                v_mu = v_id(mod(x+dxm,L), mod(y+dym,L), mod(z+dzm,L))
                v_nu = v_id(mod(x+dxn,L), mod(y+dyn,L), mod(z+dzn,L))
                rows.append(r); cols.append(e_id(v,mu)); data.append(
                rows.append(r); cols.append(e_id(v_mu,nu)); data.append(
                rows.append(r); cols.append(e_id(v_nu,mu)); data.append(
                rows.append(r); cols.append(e_id(v,nu)); data.append(
d1 = sp.csr_matrix((data,(rows,cols)), shape=(n2,n1))

# d2: F -> C (Bianchi), chosen so that d2 d1 = 0 with conventions above
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            r = c_id(v)
            v_e0 = v_id(mod(x+1,L), y, z)
            v_e1 = v_id(x, mod(y+1,L), z)
            v_e2 = v_id(x, y, mod(z+1,L))
            rows += [r, r]; cols += [f_id(v_e0,1,2), f_id(v,1,2)]; data
            rows += [r, r]; cols += [f_id(v_e1,0,2), f_id(v,0,2)]; data
            rows += [r, r]; cols += [f_id(v_e2,0,1), f_id(v,0,1)]; data
d2 = sp.csr_matrix((data,(rows,cols)), shape=(n3,n2))

return d0, d1, d2, (n0,n1,n2,n3)

# -----
# Distances between edges (L1 on edge-centers with torus wrap)
# -----

def edge_center_halfcoords(e, L):
    """
    Edge index e = 3*v + mu.
    Return center coordinates on half-lattice (integers mod 2L):
    mu=0: (2x+1, 2y, 2z)
    mu=1: (2x, 2y+1, 2z)
    mu=2: (2x, 2y, 2z+1)

```

```

    mu=1: (2^x, 2*y+1, 2*z)
    mu=2: (2*x, 2*y, 2*z+1)
    """
    v, mu = divmod(int(e), 3)
    x,y,z = inv_idx3(v, L)
    if mu == 0: return (2*x+1, 2*y, 2*z)
    if mu == 1: return (2*x, 2*y+1, 2*z)
    return (2*x, 2*y, 2*z+1)

def torus_L1_halfdist(a, b, L):
    """
    L1 distance on half-lattice with period 2L in each coord.
    Returns integer r_half = dx+dy+dz (in half-units).
    """
    P = 2*L
    r = 0
    for i in range(3):
        da = (a[i] - b[i]) % P
        da = min(da, P - da)
        r += da
    return int(r)

# -----
# Green's function decay experiment
# -----

def solve_column(M, src_idx, tol=1e-12, maxiter=5000):
    """
    Solve M g = e_src. Prefer CG; fall back to sparse direct if needed.
    """
    n = M.shape[0]
    b = np.zeros(n)
    b[src_idx] = 1.0

    # CG (SPD)
    g, info = spla.cg(M, b, rtol=tol, atol=0.0, maxiter=maxiter)
    if info == 0:
        return g

    # fallback: sparse direct
    return spla.spsolve(M, b)

def envelope_by_distance(g, L, src_edge):
    """
    Compute envelope(r) = max_{edges at distance r} |g_edge|
    using r in half-units, then report r = r_half/2.
    """
    src_c = edge_center_halfcoords(src_edge, L)
    env = {} # r_half -> maxabs

    for e in range(g.shape[0]):

```

```

        c = edge_center_halfcoords(e, L)
        r_half = torus_L1_halfdist(src_c, c, L)
        val = abs(float(g[e]))
        if r_half not in env or val > env[r_half]:
            env[r_half] = val

    r_half_sorted = np.array(sorted(env.keys()), dtype=int)
    r = r_half_sorted / 2.0
    y = np.array([env[k] for k in r_half_sorted], dtype=float)
    return r, y

def fit_decay(r, env, r_min=2.0, r_max=None, eps=1e-300):
    """
    Fit log(env) = a - c r on a chosen range.
    """
    if r_max is None:
        r_max = float(np.max(r))
    mask = (r >= r_min) & (r <= r_max) & (env > 0)
    rr = r[mask]
    yy = np.log(np.maximum(env[mask], eps))
    if rr.size < 2:
        return float("nan"), float("nan"), int(rr.size)
    # least squares slope
    A = np.vstack([np.ones_like(rr), rr]).T
    a, slope = np.linalg.lstsq(A, yy, rcond=None)[0]
    c = -float(slope)
    return float(a), float(c), int(rr.size)

def main():
    # Tunables
    L = 10          # keep modest; n1 = 3 L^3
    alpha = 1.0
    m2 = 0.30       # mass^2 in M = m^2 I + alpha d1^T d1
    src_edge = 0    # edge at v=0, mu=0

    d0, d1, d2, (n0,n1,n2,n3) = build_ops_3d_torus(L)

    # Build M on edges
    A = (d1.T @ d1).tocsr()
    M = (m2 * sp.eye(n1, format="csr")) + (alpha * A)

    # Solve one Green's function column
    g = solve_column(M, src_edge)

    # Envelope vs distance
    r, env = envelope_by_distance(g, L, src_edge)

    # Fit decay rate
    a_fit, c_fit, nfit = fit_decay(r, env, r_min=2.0, r_max=min(np.max(r), L

    m = math.sqrt(m2)

```

```

m = math.sqrt(m2)
c_pred = math.asinh(m / alpha)    # the heuristic you requested

print(f"=== Massive edge Green's function on T^3 ===")
print(f"L={L}, n1={n1}, alpha={alpha}, m^2={m2} (m={m:.6g})")
print(f"Fit range: r in [{2.0}, {min(np.max(r), L/2):.6g}] with {nfit} b")
print(f"c_fit = {c_fit:.6g}")
print(f"c_pred = asinh(m/alpha) = {c_pred:.6g}")
print()

# Print a small table of envelope values
print("r    envelope_max|g|    log(envelope)")
for rr, ee in list(zip(r, env))[:25]:
    print(f"{rr:4.1f} {ee: .3e}          {math.log(max(ee,1e-300)):+.3f}")

# And a few far points
print("\n... tail samples ...")
for rr, ee in list(zip(r, env))[-10:]:
    print(f"{rr:4.1f} {ee: .3e}          {math.log(max(ee,1e-300)):+.3f}")

if __name__ == "__main__":
    main()

```

```

=== Massive edge Green's function on T^3 ===
L=10, n1=3000, alpha=1.0, m^2=0.3 (m=0.547723)
Fit range: r in [2.0, 5] with 4 bins
c_fit = 0.825908
c_pred = asinh(m/alpha) = 0.523484

```

r	envelope_max g	log(envelope)
0.0	1.249e+00	+0.223
1.0	4.328e-01	-0.838
2.0	8.451e-02	-2.471
3.0	3.036e-02	-3.495
4.0	1.436e-02	-4.243
5.0	6.913e-03	-4.974
6.0	4.156e-03	-5.483
7.0	2.736e-03	-5.901
8.0	2.139e-03	-6.147
9.0	1.621e-03	-6.425
10.0	1.272e-03	-6.667
11.0	1.151e-03	-6.767
12.0	9.067e-04	-7.006
13.0	6.540e-04	-7.332
14.0	4.796e-04	-7.643
15.0	4.188e-04	-7.778

... tail samples ...

6.0	4.156e-03	-5.483
7.0	2.736e-03	-5.901
8.0	2.139e-03	-6.147
9.0	1.621e-03	-6.425
10.0	1.272e-03	-6.667

```

11.0  1.151e-03      -6.767
12.0  9.067e-04      -7.006
13.0  6.540e-04      -7.332
14.0  4.796e-04      -7.643
15.0  4.188e-04      -7.778

```

```

#!/usr/bin/env python3
import math
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

def idx3(x, y, z, L):
    return x + L * (y + L * z)

def inv_idx3(v, L):
    x = v % L
    v //= L
    y = v % L
    z = v // L
    return x, y, z

def mod(x, L):
    return x % L

def build_ops_3d_torus(L: int):
    n0 = L**3
    n1 = 3 * L**3
    n2 = 3 * L**3
    n3 = L**3

    pairs = [(0,1),(0,2),(1,2)]
    pair_to_id = {pairs[i]: i for i in range(3)}
    dirs = [(1,0,0),(0,1,0),(0,0,1)]

    def v_id(x,y,z): return idx3(x,y,z,L)
    def e_id(v, mu): return 3*v + mu
    def f_id(v, mu, nu):
        if mu > nu: mu, nu = nu, mu
        return 3*v + pair_to_id[(mu,nu)]

    # d0: V -> E
    rows, cols, data = [], [], []
    for z in range(L):
        for y in range(L):
            for x in range(L):
                v = v_id(x,y,z)
                for mu, (dx,dy,dz) in enumerate(dirs):
                    vp = v_id(mod(x+dx,L), mod(y+dy,L), mod(z+dz,L))
                    r = e_id(v, mu)

```

```

        rows += [r, r]
        cols += [vp, v]
        data += [1.0, -1.0]
d0 = sp.csr_matrix((data,(rows,cols)), shape=(n1,n0))

# d1: E -> F
rows, cols, data = [], [], []
for z in range(L):
    for y in range(L):
        for x in range(L):
            v = v_id(x,y,z)
            for (mu,nu) in pairs:
                r = f_id(v, mu, nu)
                dxm,dym,dzm = dirs[mu]
                dxn,dyn,dzn = dirs[nu]
                v_mu = v_id(mod(x+dxm,L), mod(y+dym,L), mod(z+dzm,L))
                v_nu = v_id(mod(x+dxn,L), mod(y+dyn,L), mod(z+dzn,L))
                rows.append(r); cols.append(e_id(v,mu)); data.append(
                rows.append(r); cols.append(e_id(v_mu,nu)); data.append(
                rows.append(r); cols.append(e_id(v_nu,mu)); data.append(
                rows.append(r); cols.append(e_id(v,nu)); data.append(
d1 = sp.csr_matrix((data,(rows,cols)), shape=(n2,n1))

return d0, d1, (n0,n1,n2,n3)

def edge_center_halfcoords(e, L):
    v, mu = divmod(int(e), 3)
    x,y,z = inv_idx3(v, L)
    if mu == 0: return (2*x+1, 2*y, 2*z)
    if mu == 1: return (2*x, 2*y+1, 2*z)
    return (2*x, 2*y, 2*z+1)

def block_id_from_halfcoords(hc, L, B):
    assert L % B == 0
    Lb = L // B
    side = 2 * B
    bx = (hc[0] // side) % Lb
    by = (hc[1] // side) % Lb
    bz = (hc[2] // side) % Lb
    return (int(bx), int(by), int(bz))

def torus_block_L1_dist(b0, b1, Lb):
    r = 0
    for i in range(3):
        da = (b0[i] - b1[i]) % Lb
        da = min(da, Lb - da)
        r += da
    return int(r)

def harmonic_basis_edges(L):
    n1 = 3 * L**3

```

```

H = []
for mu in [0,1,2]:
    h = np.zeros(n1, dtype=float)
    h[mu::3] = 1.0
    h /= np.linalg.norm(h)
    H.append(h)
return H

def project_out_harmonic(g, H):
    gp = g.copy()
    for h in H:
        gp -= (h @ gp) * h
    return gp

def solve_cg(M, b, rtol=1e-12, maxiter=20000):
    diag = M.diagonal()
    invdiag = np.where(diag != 0, 1.0/diag, 1.0)
    P = spla.LinearOperator(M.shape, matvec=lambda x: invdiag * x)
    x, info = spla.cg(M, b, M=P, rtol=rtol, atol=0.0, maxiter=maxiter)
    if info != 0:
        raise RuntimeError(f"CG did not converge (info={info}).")
    return x

def block_envelope_l2_by_Rb(g, L, src_edge, B):
    assert L % B == 0
    Lb = L // B
    src_block = block_id_from_halfcoords(edge_center_halfcoords(src_edge, L))

    block_sums = {}
    for e in range(g.shape[0]):
        bid = block_id_from_halfcoords(edge_center_halfcoords(e, L), L, B)
        block_sums[bid] = block_sums.get(bid, 0.0) + float(g[e])**2

    env = {}
    for bid, ss in block_sums.items():
        Rb = torus_block_L1_dist(src_block, bid, Lb)
        val = math.sqrt(ss)
        if Rb not in env or val > env[Rb]:
            env[Rb] = val

    Rb_sorted = np.array(sorted(env.keys()), dtype=int)
    y = np.array([env[k] for k in Rb_sorted], dtype=float)
    return Rb_sorted, y

def fit_log_linear_on_Rb(Rb, env, Rb_min, Rb_max, min_pts=3):
    mask = (Rb >= Rb_min) & (Rb <= Rb_max) & (env > 0)
    rr = Rb[mask].astype(float)
    yy = np.log(env[mask])
    if rr.size < min_pts:
        raise RuntimeError("Not enough bins in fit window.")

```

```

A = np.vstack([np.ones_like(rr), rr]).T
a, slope = np.linalg.lstsq(A, yy, rcond=None)[0]
c_per_block = -float(slope)
return float(a), float(c_per_block), int(rr.size)

def choose_fit_window(Rb, env, Lb):
    Rb_max_available = int(np.max(Rb))
    Rb_max_safe = min(int(0.45 * Lb), Rb_max_available)
    for Rb_min in [3, 2, 1]:
        try:
            a, c, n = fit_log_linear_on_Rb(Rb, env, Rb_min, Rb_max_safe, min
            return Rb_min, Rb_max_safe, a, c, n
        except RuntimeError:
            pass
    return 1, Rb_max_safe, float("nan"), float("nan"), 0

def main():
    L = 24
    B = 2
    alpha = 1.0
    m2 = 0.30
    m = math.sqrt(m2)
    src_edge = 0

    d0, d1, (n0,n1,n2,n3) = build_ops_3d_torus(L)

    A = (d1.T @ d1).tocsr()
    G = (d0 @ d0.T).tocsr()
    M = (m2 * sp.eye(n1, format="csr")) + alpha * (A + G)

    b = np.zeros(n1)
    b[src_edge] = 1.0
    g = solve_cg(M, b)

    H = harmonic_basis_edges(L)
    g = project_out_harmonic(g, H)

    Lb = L // B
    Rb, env = block_envelope_l2_by_Rb(g, L, src_edge, B)

    Rb_min, Rb_max, a_fit, c_fit_block, nfit = choose_fit_window(Rb, env, Lb)
    c_fit_lattice = c_fit_block / float(B)

    c_pred_lattice = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha)))

    print("=== Block Green's function decay for (m^2 I + alpha Δ_1)^{-1} on
    print(f"L={L}, B={B}, Lb={Lb}, n1={n1}, alpha={alpha}, m^2={m2} (m={m:.6
    print(f"Fit window in block distance: Rb in [{Rb_min}, {Rb_max}] with {n
    print(f"c_fit_block = {c_fit_block:.6g} (per block step)")
    print(f"c_fit_lattice = {c_fit_lattice:.6g} (per lattice unit)")
    print(f"c pred lattice= 2 asinh(m/(2 sqrt(alpha))) = {c pred lattice:.6g

```

```

print()
print("Rb    r=B*Rb    block_env_l2    log(env)")
for RR, ee in list(zip(Rb, env))[ :25]:
    r = B * int(RR)
    print(f"{int(RR):2d}    {r:4d}    {ee: .3e}    {math.log(max(ee,1e-3

if __name__ == "__main__":
    main()

```

```

=== Block Green's function decay for  $(m^2 I + \alpha \Delta_1)^{-1}$  on  $T^3$  ===
L=24, B=2, Lb=12, n1=41472, alpha=1.0, m^2=0.3 (m=0.547723)
Fit window in block distance: Rb in [3, 5] with 3 bins
c_fit_block    = 1.16743    (per block step)
c_fit_lattice  = 0.583715 (per lattice unit)
c_pred_lattice= 2 asinh(m/(2 sqrt(alpha))) = 0.541097

```

Rb	r=B*Rb	block_env_l2	log(env)
0	0	2.290e-01	-1.474
1	2	6.677e-02	-2.706
2	4	3.450e-02	-3.367
3	6	2.180e-02	-3.826
4	8	5.766e-03	-5.156
5	10	2.110e-03	-6.161
6	12	8.043e-04	-7.125
7	14	6.343e-04	-7.363
8	16	6.552e-04	-7.331
9	18	6.684e-04	-7.311
10	20	6.746e-04	-7.301
11	22	6.758e-04	-7.300
12	24	6.770e-04	-7.298
13	26	6.786e-04	-7.295
14	28	6.799e-04	-7.294
15	30	6.806e-04	-7.293
16	32	6.807e-04	-7.292
17	34	6.808e-04	-7.292
18	36	6.810e-04	-7.292

```

import math
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

# --- reuse your build_ops_3d_torus, idx3/inv_idx3, edge_center_halfcoords e
# Assumes you already have: build_ops_3d_torus, edge_center_halfcoords, bloc

def harmonic_basis_edges(L):
    n1 = 3 * L**3
    H = []
    for mu in [0,1,2]:
        h = np.zeros(n1, dtype=float)
        h[mu::3] = 1.0

```

```

        h /= np.linalg.norm(h)
        H.append(h)
    return H

def project_out_harmonic(g, H):
    gp = g.copy()
    for h in H:
        gp -= (h @ gp) * h
    return gp

def solve_cg(M, b, rtol=1e-12, maxiter=30000):
    diag = M.diagonal()
    invdiag = np.where(diag != 0, 1.0/diag, 1.0)
    P = spla.LinearOperator(M.shape, matvec=lambda x: invdiag * x)
    x, info = spla.cg(M, b, M=P, rtol=rtol, atol=0.0, maxiter=maxiter)
    if info != 0:
        raise RuntimeError(f"CG did not converge (info={info}).")
    return x

def block_envelope_l2_by_Rb(g, L, src_edge, B):
    assert L % B == 0
    Lb = L // B
    src_block = block_id_from_halfcoords(edge_center_halfcoords(src_edge, L))

    block_sums = {}
    for e in range(g.shape[0]):
        bid = block_id_from_halfcoords(edge_center_halfcoords(e, L), L, B)
        block_sums[bid] = block_sums.get(bid, 0.0) + float(g[e])**2

    env = {}
    for bid, ss in block_sums.items():
        Rb = torus_block_L1_dist(src_block, bid, Lb)
        val = math.sqrt(ss)
        if Rb not in env or val > env[Rb]:
            env[Rb] = val

    Rb_sorted = np.array(sorted(env.keys()), dtype=int)
    y = np.array([env[k] for k in Rb_sorted], dtype=float)
    return Rb_sorted, y

def local_slopes(r, env):
    # r, env arrays; return slopes at midpoints
    rr = r.astype(float)
    yy = np.log(np.maximum(env, 1e-300))
    dr = rr[1:] - rr[:-1]
    slopes = -(yy[1:] - yy[:-1]) / dr
    mids = 0.5*(rr[1:] + rr[:-1])
    return mids, slopes

def max_offdiag_row_sum(K: sp.csr_matrix):
    # bound for CT-tvne estimates: max i sum {i!=j} |K[i,j]|

```

```

K = K.tocsr()
absK = abs(K)
diag = absK.diagonal()
absK = absK - sp.diags(diag, format="csr")
row_sums = np.array(absK.sum(axis=1)).ravel()
return float(np.max(row_sums))

# ----- run -----

L = 24
B = 1          # <-- KEY: many bins
alpha = 1.0
m2 = 0.30
m = math.sqrt(m2)
src_edge = 0

d0, d1, (n0,n1,n2,n3) = build_ops_3d_torus(L)

A = (d1.T @ d1).tocsr()
G = (d0 @ d0.T).tocsr()
Delta1 = (A + G).tocsr()

M = (m2 * sp.eye(n1, format="csr")) + alpha * Delta1

b = np.zeros(n1); b[src_edge] = 1.0
g = solve_cg(M, b)

# remove the 3 harmonic flows
g = project_out_harmonic(g, harmonic_basis_edges(L))

Rb, env = block_envelope_l2_by_Rb(g, L, src_edge, B)
r = (Rb * B).astype(float)

# compute local slopes and take a robust mid-range plateau
mids, slopes = local_slopes(r, env)

# choose a fit window safely below L/2
r_min = 4.0
r_max = 0.40 * L
mask = (mids >= r_min) & (mids <= r_max)
c_med = float(np.median(slopes[mask]))
c_mean = float(np.mean(slopes[mask]))

# your heuristic prediction (one reasonable discrete choice)
c_pred = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha)))

# conservative CT-style bound using row-sum of the finite-range part
C0 = max_offdiag_row_sum(Delta1)
c_ct = math.log(1.0 + m2 / (2.0 * alpha * C0)) # very conservative, but exp

```

```

print("=== Decay diagnostics ===")
print(f"L={L}, B={B}, alpha={alpha}, m^2={m2} (m={m:.6g}), n1={n1}")
print(f"mid-range window: r in [{r_min}, {r_max}]")
print(f"c_local median = {c_med:.6g}")
print(f"c_local mean   = {c_mean:.6g}")
print(f"c_pred          = {c_pred:.6g}")
print(f"C0 (max offdiag row-sum of Δ1) = {C0:.6g}")
print(f"c_CT (conservative)              = {c_ct:.6g}")

print("\nFirst few envelope points:")
for RR, ee in list(zip(Rb, env))[1:12]:
    print(f"Rb={int(RR):2d}, r={int(RR)*B:2d}, env={ee:.3e}, log={math.log(m

```

```

=== Decay diagnostics ===
L=24, B=1, alpha=1.0, m^2=0.3 (m=0.547723), n1=41472
mid-range window: r in [4.0, 9.600000000000001]
c_local median = 0.523138
c_local mean   = 0.545397
c_pred        = 0.541097
C0 (max offdiag row-sum of Δ1) = 6
c_CT (conservative)              = 0.0246926

```

```

First few envelope points:
Rb= 0, r= 0, env=2.069e-01, log=-1.576
Rb= 1, r= 1, env=5.059e-02, log=-2.984
Rb= 2, r= 2, env=2.411e-02, log=-3.725
Rb= 3, r= 3, env=1.512e-02, log=-4.192
Rb= 4, r= 4, env=7.665e-03, log=-4.871
Rb= 5, r= 5, env=4.530e-03, log=-5.397
Rb= 6, r= 6, env=2.934e-03, log=-5.831
Rb= 7, r= 7, env=1.656e-03, log=-6.403
Rb= 8, r= 8, env=9.844e-04, log=-6.923
Rb= 9, r= 9, env=5.957e-04, log=-7.426
Rb=10, r=10, env=2.906e-04, log=-8.144
Rb=11, r=11, env=2.165e-04, log=-8.438

```

```

#!/usr/bin/env python3
# ct_decay_gpu.py
import math
import torch

# -----
# Utilities
# -----

def torus_l1_dist_grid(L, d, device):
    """dist[x] = sum_i min(x_i, L-x_i) on T^d, returned as int tensor shape
    coords = torch.meshgrid(*[torch.arange(L, device=device) for _ in range(
    dist = torch.zeros([L]*d, dtype=torch.int32, device=device)
    for ax in range(d):
        x = coords[ax]

```

```

        dist += torch.minimum(x, (L - x) % L).to(torch.int32)
    return dist

def env_by_dist(blocknorm, dist, r_max):
    """
    blocknorm: tensor shape [L]*d (float)
    dist: int tensor same shape
    returns env[r] = max_{dist==r} blocknorm
    """
    env = torch.zeros(r_max+1, device=blocknorm.device, dtype=blocknorm.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(blocknorm[m])
    return env

def local_slopes(env):
    """slopes[r+0.5] = -(log env[r+1]-log env[r])"""
    eps = 1e-300
    y = torch.log(torch.clamp(env, min=eps))
    dy = y[1:] - y[:-1]
    slopes = -dy # delta r = 1
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device)
    return mids, slopes

# -----
# Operator:  $\Delta 1$  on 1-forms is componentwise scalar Laplacian
#  $\Delta X_\mu = \sum_{\nu=1..d} (2 X_\mu - \text{shift}^+ - \text{shift}^-)$ 
# -----

def laplacian_componentwise(X):
    # X shape: [d, L, ..., L]
    d = X.shape[0]
    Y = torch.zeros_like(X)
    for ax in range(1, X.ndim): # spatial axes
        Y += (2.0 * X - torch.roll(X, shifts=+1, dims=ax) - torch.roll(X, sh
    return Y

def apply_K(X, R_power):
    """
    K = ( $\Delta 1$ )^R_power. Range = R_power (in the vertex L1 metric).
    """
    Y = X
    for _ in range(R_power):
        Y = laplacian_componentwise(Y)
    return Y

# -----
# Matrix-free CG for  $(m^2 I + \alpha K) g = b$ 
# -----

```

```

def cg_solve(matvec, b, x0=None, rtol=1e-10, maxiter=20000):
    if x0 is None:
        x = torch.zeros_like(b)
    else:
        x = x0.clone()

    r = b - matvec(x)
    p = r.clone()
    rsold = torch.sum(r*r)

    bnorm = torch.sqrt(torch.sum(b*b))
    if bnorm == 0:
        return x

    for it in range(maxiter):
        Ap = matvec(p)
        alpha = rsold / torch.sum(p*Ap)
        x = x + alpha*p
        r = r - alpha*Ap
        rsnew = torch.sum(r*r)
        if torch.sqrt(rsnew) <= rtol * bnorm:
            return x
        p = r + (rsnew/rsold)*p
        rsold = rsnew

    raise RuntimeError("CG did not converge")

# -----
# Harmonic projection: remove constant 1-forms (d-dimensional harmonic secto
# -----

def project_out_harmonic(g):
    # subtract global mean per component
    mean = g.mean(dim=tuple(range(1, g.ndim)), keepdim=True)
    return g - mean

# -----
# Compute exact C0 for translation-invariant K by applying it to delta sourc
# C0 = sum_{y != 0} ||K_{0y}||_op. Here operator is diagonal in Lie componen
# so it's just sum_{y != 0} |kernel(y)| for one component.
# -----

def compute_C0(K_apply_on_vec, shape_spatial, device):
    d = len(shape_spatial)
    L = shape_spatial[0]
    X = torch.zeros((1,)+shape_spatial, device=device) # one component
    # delta at origin
    X[(0,)+(0,)*d] = 1.0
    Y = K_apply_on_vec(X)[0] # kernel over sites
    Y_abs = torch.abs(Y)

```

```

    Y_abs[(0,)*d] = 0.0
    return float(Y_abs.sum().item())

# -----
# Main experiment
# -----

def run_experiment(
    d=4,
    L=32,
    m2=0.3,
    alpha=1.0,
    R_power=1,          # K = ( $\Delta 1$ )^R_power, true range R_power
    r_window=(4, None),
    device="cuda",
):
    device = device if (device == "cuda" and torch.cuda.is_available()) else
    torch.set_default_dtype(torch.float64)

    # unknowns g live on 1-forms: shape [d, L,...,L]
    spatial = (L,)*d
    shape = (d,)+spatial

    def K_mv(X):
        return apply_K(X, R_power)

    def M_mv(X):
        return m2 * X + alpha * K_mv(X)

    # RHS: delta in component 0 at origin
    b = torch.zeros(shape, device=device)
    b[(0,)+(0,)*d] = 1.0

    g = cg_solve(M_mv, b, rtol=1e-11, maxiter=50000)
    g = project_out_harmonic(g)

    # blocknorm per vertex-block (B=1): sqrt(sum_mu g_mu(x)^2)
    blocknorm = torch.sqrt(torch.sum(g*g, dim=0))

    # dist grid and envelope
    dist = torus_l1_dist_grid(L, d, device=device)
    r_max = (L//2) * d # absolute max L1 distance on torus
    env = env_by_dist(blocknorm, dist, r_max)

    mids, slopes = local_slopes(env)

    r_min = r_window[0]
    r_max_fit = r_window[1]
    if r_max_fit is None:
        # safe mid-range below wraparound: ~0.4*L*d is too big in d dims; us
        r_max_fit = int(0.35 * L)

```

```

mask = (mids >= r_min) & (mids <= r_max_fit) & torch.isfinite(slopes)
c_med = float(torch.median(slopes[mask]).item())
c_mean = float(torch.mean(slopes[mask]).item())

# constants
m = math.sqrt(m2)
# 1D exact decay for  $m^2 I + \alpha \Delta$ :  $\kappa = \text{arcosh}(1+m^2/(2\alpha)) = 2 \text{ asinh}(m / (2.0 * \mathbf{sqrt}(\alpha)))$ 
c_pred_1d = 2.0 * math.asinh(m / (2.0 * math.sqrt(alpha)))

# exact C0 for K
C0 = compute_C0(lambda X: apply_K(X, R_power), spatial, device=device)

# CT-style conservative rate in vertex metric with range R_power:
eta_CT = (1.0 / R_power) * math.log(1.0 + m2 / (2.0 * alpha * C0))

print("=== CT decay experiment (matrix-free, GPU-friendly) ===")
print(f"d={d}, L={L}, unknowns={d}*L^d={d*(L**d)}")
print(f"Operator: M = m^2 I + alpha K, K = (\Delta 1)^{R_power} (range R={R_p")
print(f"m^2={m2}, alpha={alpha}, m={m:.6g}")
print(f"Fit window mids in [{r_min}, {r_max_fit}] (vertex L1 metric)")
print(f"c_local median = {c_med:.6g}")
print(f"c_local mean = {c_mean:.6g}")
print(f"c_pred_1d = {c_pred_1d:.6g} (benchmark only)")
print(f"C0 (exact row-sum for K) = {C0:.6g}")
print(f"eta_CT (conservative) = {eta_CT:.6g}")
print()

# print a short envelope table
for r in range(0, 25):
    if r < env.numel():
        val = float(env[r].item())
        logv = math.log(max(val, 1e-300))
        print(f"r={r:2d} env={val:.3e} log={logv:+.3f}")
return {
    "c_med": c_med,
    "c_mean": c_mean,
    "c_pred_1d": c_pred_1d,
    "C0": C0,
    "eta_CT": eta_CT,
}

if __name__ == "__main__":
    # Recommended A100 runs:
    # 1) d=4, R=1 (your actual  $\Delta 1$ ): confirms C0=8 and stable mid-range slope
    run_experiment(d=4, L=32, m2=0.3, alpha=1.0, R_power=1, r_window=(6, 18))

    # 2) range stress-test:  $K=(\Delta 1)^R$  (true range R). This mimics coarse-grai
    # Expect: C0 grows with R; eta_CT shrinks  $\sim 1/R$ ; measured slope stays
    run_experiment(d=4, L=32, m2=0.3, alpha=1.0, R_power=2, r_window=(6, 18))

```

```
run_experiment(d=4, L=32, m2=0.3, alpha=1.0, R_power=3, r_window=(6, 18))
```

```
=== CT decay experiment (matrix-free, GPU-friendly) ===
```

```
d=4, L=32, unknowns=4*L^d=4194304
```

```
Operator:  $M = m^2 I + \alpha K$ ,  $K = (\Delta 1)^1$  (range  $R=1$ )
```

```
 $m^2=0.3$ ,  $\alpha=1.0$ ,  $m=0.547723$ 
```

```
Fit window mids in [6, 18] (vertex L1 metric)
```

```
c_local median = 0.440021
```

```
c_local mean = 0.463479
```

```
c_pred_1d = 0.541097 (benchmark only)
```

```
 $C0$  (exact row-sum for  $K$ ) = 8
```

```
eta_CT (conservative) = 0.0185764
```

```
r= 0 env=1.439e-01 log=-1.939
```

```
r= 1 env=2.426e-02 log=-3.719
```

```
r= 2 env=8.720e-03 log=-4.742
```

```
r= 3 env=4.529e-03 log=-5.397
```

```
r= 4 env=2.856e-03 log=-5.858
```

```
r= 5 env=1.396e-03 log=-6.574
```

```
r= 6 env=7.944e-04 log=-7.138
```

```
r= 7 env=5.000e-04 log=-7.601
```

```
r= 8 env=3.374e-04 log=-7.994
```

```
r= 9 env=2.003e-04 log=-8.516
```

```
r=10 env=1.269e-04 log=-8.972
```

```
r=11 env=8.437e-05 log=-9.380
```

```
r=12 env=5.810e-05 log=-9.753
```

```
r=13 env=3.642e-05 log=-10.220
```

```
r=14 env=2.346e-05 log=-10.660
```

```
r=15 env=1.532e-05 log=-11.086
```

```
r=16 env=1.002e-05 log=-11.511
```

```
r=17 env=5.707e-06 log=-12.074
```

```
r=18 env=3.052e-06 log=-12.700
```

```
r=19 env=3.065e-06 log=-12.695
```

```
r=20 env=3.082e-06 log=-12.690
```

```
r=21 env=3.102e-06 log=-12.684
```

```
r=22 env=3.119e-06 log=-12.678
```

```
r=23 env=3.134e-06 log=-12.673
```

```
r=24 env=3.146e-06 log=-12.669
```

```
=== CT decay experiment (matrix-free, GPU-friendly) ===
```

```
d=4, L=32, unknowns=4*L^d=4194304
```

```
Operator:  $M = m^2 I + \alpha K$ ,  $K = (\Delta 1)^2$  (range  $R=2$ )
```

```
 $m^2=0.3$ ,  $\alpha=1.0$ ,  $m=0.547723$ 
```

```
Fit window mids in [6, 18] (vertex L1 metric)
```

```
c_local median = 0.232472
```

```
c_local mean = 0.37847
```

```
c_pred_1d = 0.541097 (benchmark only)
```

```
 $C0$  (exact row-sum for  $K$ ) = 184
```

```
eta_CT (conservative) = 0.000407443
```

```
r= 0 env=3.338e-02 log=-3.400
```

```
r= 1 env=1.481e-02 log=-4.213
```

```
r= 2 env=9.039e-03 log=-4.706
```

```
r= 3 env=6.378e-03 log=-5.055
```

```
r= 4 env=4.846e-03 log=-5.330
```

```
r= 5 env=3.800e-03 log=-5.675
```

```

r= 5  env=2.809e-03  log=-5.875
r= 6  env=1.738e-03  log=-6.355
r= 7  env=1.118e-03  log=-6.796
r= 8  env=7.349e-04  log=-7.216
r= 9  env=3.851e-04  log=-7.862
r=10  env=1.958e-04  log=-8.538

```

Start coding or [generate](#) with AI.

```

#!/usr/bin/env python3
import argparse, math
import numpy as np
import torch

# -----
# Torch utilities
# -----

def torus_L1_dist_grid(L, D, device):
    coords = torch.meshgrid(*[torch.arange(L, device=device) for _ in range(
        dist = torch.zeros([L]*D, dtype=torch.int32, device=device)
    for ax in range(D):
        x = coords[ax]
        dist += torch.minimum(x, (L - x) % L).to(torch.int32)
    return dist

def env_by_dist(blocknorm, dist, r_max):
    env = torch.zeros(r_max+1, device=blocknorm.device, dtype=blocknorm.dtyp
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(blocknorm[m])
    return env

def local_slopes(env):
    eps = 1e-300
    y = torch.log(torch.clamp(env, min=eps))
    slopes = -(y[1:] - y[:-1]) # Δr=1
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device)
    return mids, slopes

def block_l2_norm(g, B):
    """
    g: [D, L, ..., L] with D spatial dims and D components (1-forms)
    returns blocknorm on blocks of side B: [Lb, ..., Lb]
    """
    D = g.shape[0]
    L = g.shape[1]
    assert all(g.shape[i] == L for i in range(1, 1+D))
    assert L % B == 0
    Lb = L // B

```

```

    Lb = L // B

    shape = [D]
    for _ in range(D):
        shape += [Lb, B]
    gg = g.reshape(shape)

    # sum over component axis 0 and all the "B" axes (2,4,...,2D)
    sum_axes = [0] + [2*i+2 for i in range(D)]
    s = (gg*gg).sum(dim=sum_axes)
    return torch.sqrt(s) # [Lb,...,Lb]

def project_out_constant_1forms(g):
    # subtract global mean per component
    mean = g.mean(dim=tuple(range(1, g.ndim)), keepdim=True)
    return g - mean

# -----
# Preconditioned CG (matrix-free)
# -----

@torch.no_grad()
def pcg(matvec, b, invdiag_comp, rtol=1e-11, maxiter=60000):
    """
    Solve Mx=b with PCG. invdiag_comp: tensor [D] giving Jacobi inverse diag
    Broadcasted over sites.
    """
    x = torch.zeros_like(b)
    r = b - matvec(x)

    z = invdiag_comp.view(-1, *[1]*(b.ndim-1)) * r
    p = z.clone()
    rz_old = torch.sum(r*z)

    bnorm = torch.sqrt(torch.sum(b*b))
    if float(bnorm) == 0.0:
        return x

    for it in range(maxiter):
        Ap = matvec(p)
        alpha = rz_old / torch.sum(p*Ap)
        x = x + alpha*p
        r = r - alpha*Ap
        if torch.sqrt(torch.sum(r*r)) <= rtol * bnorm:
            return x
        z = invdiag_comp.view(-1, *[1]*(b.ndim-1)) * r
        rz_new = torch.sum(r*z)
        beta = rz_new / rz_old
        p = z + beta*p
        rz_old = rz_new

```

```

        raise RuntimeError("PCG did not converge")

# -----
# Stencil representation: K acts on 1-forms as
#  $(K X)_\mu(x) = \sum_s \text{shift } s \sum_\nu A[s_i, \mu, \nu] * X_\nu(x + s)$ 
# shifts: [S,D] int
# mats:    [S,D,D] float
# -----

def load_stencil_npz(path, device, dtype):
    dat = np.load(path)
    shifts = dat["shifts"].astype(np.int64)          # [S,D]
    mats    = dat["mats"].astype(np.float64)         # [S,D,D]
    shifts_t = torch.tensor(shifts, device=device)
    mats_t   = torch.tensor(mats, device=device, dtype=dtype)
    return shifts_t, mats_t

def apply_stencil(X, shifts, mats):
    """
    X: [D, L,...,L]
    shifts: [S,D]
    mats: [S,D,D]
    """
    D = X.shape[0]
    spatial_dims = tuple(range(1, 1+D))
    Y = torch.zeros_like(X)
    for i in range(shifts.shape[0]):
        sh = tuple(int(s.item()) for s in shifts[i])
        Xs = torch.roll(X, shifts=sh, dims=spatial_dims)
        Y = Y + torch.einsum("mn,n...->m...", mats[i], Xs)
    return Y

def signed_shift_from_coords(coords, L):
    # coords in [0..L-1], map to signed in [-L//2..L//2]
    out = []
    for c in coords:
        c = int(c)
        out.append(c if c <= L//2 else c - L)
    return tuple(out)

@torch.no_grad()
def extract_translation_invariant_stencil(K_mv_user, D, L_small, tol=1e-12,
    """
    Probes K_mv_user on a small torus to extract a translation-invariant ste
    Returns shifts [S,D], mats [S,D,D] on CPU numpy arrays.
    """
    device = device if (device == "cuda" and torch.cuda.is_available()) else
    dtype = torch.float64

    # dict: shift -> matrix [D,D]
    accum = {}

```

```

    accum = {}

# precompute all site coords (CPU loop is fine for L_small <= 10)
grid = np.indices((L_small,)*D).reshape(D, -1).T # [L^D, D]

for nu in range(D):
    X = torch.zeros((D,) + (L_small,)*D, device=device, dtype=dtype)
    X[(nu,) + (0,)*D] = 1.0
    Y = K_mv_user(X).detach().cpu().numpy() # [D, L,...,L]

    for idx in range(grid.shape[0]):
        coords = grid[idx]
        sh = signed_shift_from_coords(coords, L_small)
        # read Y[:, coords]
        sl = (slice(None),) + tuple(int(c) for c in coords)
        yvec = Y[sl] # [D]
        # store coefficients for each output mu
        if np.max(np.abs(yvec)) <= tol:
            continue
        if sh not in accum:
            accum[sh] = np.zeros((D, D), dtype=np.float64)
        for mu in range(D):
            accum[sh][mu, nu] = yvec[mu]

# pack
shifts = np.array(sorted(accum.keys()), dtype=np.int64) # [S,D]
mats = np.stack([accum[tuple(sh)] for sh in shifts], axis=0) # [S,D,D]
return shifts, mats

def compute_R_and_C0(shifts, mats):
    """
    Compute:
    - R_move: max L1(|shift|) over shift!=0 with any nonzero
    - C0_move: max_row sum_{shift!=0, nu} |A_shift[mu,nu]|
    Also returns diag_comp from shift=0 (if present).
    """
    # shifts: [S,D] torch int
    # mats: [S,D,D] torch float
    D = mats.shape[1]
    S = shifts.shape[0]

    # find shift==0 index if any
    zero = torch.zeros((shifts.shape[1],), device=shifts.device, dtype=shifts.dtype)
    is0 = torch.all(shifts == zero.view(1,-1), dim=1)
    if torch.any(is0):
        i0 = int(torch.where(is0)[0][0].item())
        diag_comp = torch.diagonal(mats[i0], 0) # [D]
    else:
        diag_comp = torch.zeros(D, device=mats.device, dtype=mats.dtype)

    # movement mask (shift != 0)

```

```

l1 = torch.sum(torch.abs(shifts), dim=1) # [S]
move = (l1 > 0)

# R_move
R_move = int(torch.max(l1[move]).item()) if torch.any(move) else 0

# C0_move: max over row mu of sum_{move shifts, nu} |A|
Aabs = torch.abs(mats) # [S,D,D]
Aabs_move = Aabs[move] if torch.any(move) else Aabs[:0]
if Aabs_move.shape[0] == 0:
    C0_move = 0.0
else:
    row_sums = torch.sum(Aabs_move, dim=(0,2)) # [D] sum over shifts an
    C0_move = float(torch.max(row_sums).item())

return R_move, C0_move, diag_comp

# -----
# Decay experiment (block metric)
# -----

@torch.no_grad()
def run_decay(
    D, L, B, m2, alpha,
    shifts, mats,
    rmin, rmax,
    device="cuda",
    rtol=1e-11,
    maxiter=60000
):
    device = device if (device == "cuda" and torch.cuda.is_available()) else
    dtype = torch.float64

    shifts = shifts.to(device)
    mats = mats.to(device, dtype=dtype)

    # constants
    R_move, C0_move, diag_comp = compute_R_and_C0(shifts, mats)
    invdiag_comp = 1.0 / (m2 + alpha * diag_comp)

    eta_CT = (0.0 if (R_move == 0 or C0_move == 0.0)
              else (1.0 / R_move) * math.log(1.0 + m2 / (2.0 * alpha * C0_mo

    # operator M = m2 I + alpha K
    def K_mv(X):
        return apply_stencil(X, shifts, mats)

    def M_mv(X):
        return m2 * X + alpha * K_mv(X)

    # RHS: delta in component 0 at origin

```

```

.. ----- in components of g-----
b = torch.zeros((D,) + (L,)*D, device=device, dtype=dtype)
b[(0,) + (0,)*D] = 1.0

g = pcg(M_mv, b, invdiag_comp.to(device, dtype=dtype), rtol=rtol, maxite
g = project_out_constant_1forms(g)

# block envelope in block metric
blocknorm = block_l2_norm(g, B) # [Lb,...,Lb]
Lb = L // B
dist = torus_L1_dist_grid(Lb, D, device=device) # [Lb,...,Lb]
r_max = int(D * (Lb//2))
env = env_by_dist(blocknorm, dist, r_max)

mids, slopes = local_slopes(env)
if rmax is None:
    rmax = int(0.45 * Lb)
mask = (mids >= rmin) & (mids <= rmax) & torch.isfinite(slopes)
c_med = float(torch.median(slopes[mask]).item())
c_mean = float(torch.mean(slopes[mask]).item())

# print
print("=== Coarse-operator CT constants + measured decay ===")
print(f"D={D}, L={L}, B={B}, Lb={Lb}, unknowns={D*(L*D)}")
print(f"m^2={m2}, alpha={alpha}")
print(f"R_move = {R_move}")
print(f"C0_move = {C0_move:.6g}")
print(f"eta_CT = {eta_CT:.6g}")
print(f"fit window (block dist mids): [{rmin}, {rmax}]")
print(f"c_local median = {c_med:.6g}")
print(f"c_local mean = {c_mean:.6g}")
print()

# small table
for r in range(0, min(r_max, 30)+1):
    val = float(env[r].item())
    logv = math.log(max(val, 1e-300))
    print(f"r={r:2d} env={val:.3e} log={logv:+.3f}")

return dict(R_move=R_move, C0_move=C0_move, eta_CT=eta_CT, c_med=c_med,

# -----
# Main
# -----

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--stencil_npz", type=str, default=None,
                    help="NPZ with arrays: shifts [S,D] int, mats [S,D,D] fl
    ap.add_argument("--save_stencil_npz", type=str, default="coarse_stencil.
                    help="Where to save extracted stencil (extract mode)")

```

```

ap.add_argument("--extract", action="store_true",
                help="Extract stencil by probing K_mv_user on small L (y
ap.add_argument("--L_extract", type=int, default=8)
ap.add_argument("--tol", type=float, default=1e-12)

ap.add_argument("--D", type=int, default=4)
ap.add_argument("--L", type=int, default=32)
ap.add_argument("--B", type=int, default=1)

ap.add_argument("--m2", type=float, default=0.3)
ap.add_argument("--alpha", type=float, default=1.0)

ap.add_argument("--rmin", type=float, default=6.0)
ap.add_argument("--rmax", type=float, default=None)

ap.add_argument("--device", type=str, default="cuda")
ap.add_argument("--rtol", type=float, default=1e-11)
ap.add_argument("--maxiter", type=int, default=60000)

args = ap.parse_args()

device = args.device if (args.device == "cuda" and torch.cuda.is_availab
torch.set_default_dtype(torch.float64)

if args.extract:
    # -----
    # YOU implement this for your real coarse operator on 1-forms:
    # input/output shape: [D, L, ..., L]
    # -----
    def K_mv_user(X):
        raise NotImplementedError("Implement your coarse operator matvec

    shifts, mats = extract_translation_invariant_stencil(
        K_mv_user, D=args.D, L_small=args.L_extract, tol=args.tol, devic
    )
    np.savez(args.save_stencil_npz, shifts=shifts, mats=mats)
    print(f"Saved extracted stencil to {args.save_stencil_npz}")
    return

if args.stencil_npz is None:
    raise SystemExit("Provide --stencil_npz <file.npz> OR run with --ext

shifts_t, mats_t = load_stencil_npz(args.stencil_npz, device=device, dt

run_decay(
    D=args.D, L=args.L, B=args.B,
    m2=args.m2, alpha=args.alpha,
    shifts=shifts_t, mats=mats_t,
    rmin=args.rmin, rmax=args.rmax,
    device=device, rtol=args.rtol, maxiter=args.maxiter
)

```

```
if __name__ == "__main__":
    main()
```

```
usage: colab_kernel_launcher.py [-h] [--stencil_npz STENCIL_NPZ]
                                [--save_stencil_npz SAVE_STENCIL_NPZ]
                                [--extract] [--L_extract L_EXTRACT]
                                [--tol TOL] [--D D] [--L L] [--B B] [--m2 M2]
                                [--alpha ALPHA] [--rmin RMIN] [--rmax RMAX]
                                [--device DEVICE] [--rtol RTOL]
                                [--maxiter MAXITER]
```

colab_kernel_launcher.py: error: unrecognized arguments: -f /root/.local/share
An exception has occurred, use %tb to see the full traceback.

SystemExit: 2

/usr/local/lib/python3.12/dist-packages/IPython/core/interactiveshell.py:3561
warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)

Next steps: [Explain error](#)

Start coding or [generate](#) with AI.

Put this in a notebook cell (NOT argparse). It runs the same harness.

```
import math
import numpy as np
import torch
```

```
torch.set_default_dtype(torch.float64)
```

```
# -----
# Torch utilities
# -----
```

```
def torus_L1_dist_grid(L, D, device):
    coords = torch.meshgrid(*[torch.arange(L, device=device) for _ in range(
    dist = torch.zeros([L]*D, dtype=torch.int32, device=device)
    for ax in range(D):
        x = coords[ax]
        dist += torch.minimum(x, (L - x) % L).to(torch.int32)
    return dist
```

```
def env_by_dist(blocknorm, dist, r_max):
    env = torch.zeros(r_max+1, device=blocknorm.device, dtype=blocknorm.dtyp
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(blocknorm[m])
```

```

        env[1:] = torch.clamp(env, min=eps)
    return env

def local_slopes(env):
    eps = 1e-300
    y = torch.log(torch.clamp(env, min=eps))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device)
    return mids, slopes

def block_l2_norm(g, B):
    D = g.shape[0]
    L = g.shape[1]
    assert all(g.shape[i] == L for i in range(1, 1+D))
    assert L % B == 0
    Lb = L // B
    shape = [D]
    for _ in range(D):
        shape += [Lb, B]
    gg = g.reshape(shape)
    sum_axes = [0] + [2*i+2 for i in range(D)]
    s = (gg*gg).sum(dim=sum_axes)
    return torch.sqrt(s)

def project_out_constant_1forms(g):
    mean = g.mean(dim=tuple(range(1, g.ndim)), keepdim=True)
    return g - mean

@torch.no_grad()
def pcg(matvec, b, invdiag_comp, rtol=1e-11, maxiter=60000):
    x = torch.zeros_like(b)
    r = b - matvec(x)
    z = invdiag_comp.view(-1, *[1]*(b.ndim-1)) * r
    p = z.clone()
    rz_old = torch.sum(r*z)
    bnorm = torch.sqrt(torch.sum(b*b))
    if float(bnorm) == 0.0:
        return x
    for _ in range(maxiter):
        Ap = matvec(p)
        alpha = rz_old / torch.sum(p*Ap)
        x = x + alpha*p
        r = r - alpha*Ap
        if torch.sqrt(torch.sum(r*r)) <= rtol * bnorm:
            return x
        z = invdiag_comp.view(-1, *[1]*(b.ndim-1)) * r
        rz_new = torch.sum(r*z)
        beta = rz_new / rz_old
        p = z + beta*p
        rz_old = rz_new
    raise RuntimeError("PCG did not converge")

```

```

# -----
# Stencil representation
# -----

def apply_stencil(X, shifts, mats):
    D = X.shape[0]
    spatial_dims = tuple(range(1, 1+D))
    Y = torch.zeros_like(X)
    for i in range(shifts.shape[0]):
        sh = tuple(int(s.item()) for s in shifts[i])
        Xs = torch.roll(X, shifts=sh, dims=spatial_dims)
        Y = Y + torch.einsum("mn,n...->m...", mats[i], Xs)
    return Y

def compute_R_and_C0(shifts, mats):
    D = mats.shape[1]
    zero = torch.zeros((shifts.shape[1],), device=shifts.device, dtype=shifts.dtype)
    is0 = torch.all(shifts == zero.view(1,-1), dim=1)
    if torch.any(is0):
        i0 = int(torch.where(is0)[0][0].item())
        diag_comp = torch.diagonal(mats[i0], 0)
    else:
        diag_comp = torch.zeros(D, device=mats.device, dtype=mats.dtype)

    l1 = torch.sum(torch.abs(shifts), dim=1)
    move = (l1 > 0)
    R_move = int(torch.max(l1[move]).item()) if torch.any(move) else 0

    Aabs = torch.abs(mats)
    Aabs_move = Aabs[move] if torch.any(move) else Aabs[:0]
    if Aabs_move.shape[0] == 0:
        C0_move = 0.0
    else:
        row_sums = torch.sum(Aabs_move, dim=(0,2)) # [D]
        C0_move = float(torch.max(row_sums).item())

    return R_move, C0_move, diag_comp

def load_stencil_npz(path, device):
    dat = np.load(path)
    shifts = torch.tensor(dat["shifts"].astype(np.int64), device=device)
    mats = torch.tensor(dat["mats"].astype(np.float64), device=device, dtype=torch.float64)
    return shifts, mats

# -----
# Run decay test for a provided stencil NPZ
# -----

@torch.no_grad()
def run_decay_from_npz(stencil_npz, D=4, l=32, R=1, m2=0.3, alpha=1.0,

```

```

def apply_decay_from_npz(stencil_npz, rmin, rmax, rmax, m2, alpha, rmax,
                        rmin=6.0, rmax=18.0, device="cuda", rtol=1e-11, maxiter=100,
                        device = device if (device == "cuda" and torch.cuda.is_available()) else "cpu",
                        shifts, mats = load_stencil_npz(stencil_npz, device=device)

R_move, C0_move, diag_comp = compute_R_and_C0(shifts, mats)
invdiag_comp = 1.0 / (m2 + alpha * diag_comp)

eta_CT = (0.0 if (R_move == 0 or C0_move == 0.0)
           else (1.0 / R_move) * math.log(1.0 + m2 / (2.0 * alpha * C0_move)))

def K_mv(X):
    return apply_stencil(X, shifts, mats)

def M_mv(X):
    return m2 * X + alpha * K_mv(X)

b = torch.zeros((D,) + (L,)*D, device=device)
b[(0,) + (0,)*D] = 1.0

g = pcg(M_mv, b, invdiag_comp, rtol=rtol, maxiter=maxiter)
g = project_out_constant_1forms(g)

blocknorm = block_l2_norm(g, B)
Lb = L // B
dist = torus_L1_dist_grid(Lb, D, device=device)
r_max = int(D * (Lb//2))
env = env_by_dist(blocknorm, dist, r_max)

mids, slopes = local_slopes(env)
mask = (mids >= rmin) & (mids <= rmax) & torch.isfinite(slopes)
c_med = float(torch.median(slopes[mask])).item()
c_mean = float(torch.mean(slopes[mask])).item()

print("=== Coarse-operator CT constants + measured decay (notebook) ===")
print(f"stencil={stencil_npz}")
print(f"D={D}, L={L}, B={B}, Lb={Lb}, unknowns={D*(L*D)}")
print(f"m^2={m2}, alpha={alpha}")
print(f"R_move={R_move}")
print(f"C0_move={C0_move:.6g}")
print(f"eta_CT={eta_CT:.6g}")
print(f"fit window mids in [{rmin}, {rmax}]")
print(f"c_local median={c_med:.6g}")
print(f"c_local mean={c_mean:.6g}")
print()
for r in range(0, min(r_max, 30)+1):
    val = float(env[r].item())
    logv = math.log(max(val, 1e-300))
    print(f"r={r:2d} env={val:.3e} log={logv:+.3f}")

return dict(R_move=R_move, C0_move=C0_move, eta_CT=eta_CT, c_med=c_med,

```

```
# -----
# USAGE:
# -----
# 1) Put your stencil file somewhere accessible, e.g. "coarse_stencil.npz"
# 2) Then run:
# run_decay_from_npz("coarse_stencil.npz", D=4, L=32, B=1, m2=0.3, alpha=1.0
```

Notebook cell: exact (FFT) Green's function decay for $M = m^2 I + \alpha (\Delta$
 # No placeholders. No user "drop-ins". Runs on A100. Prints ledger constants

```
import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# -----
# Core lattice objects on  $T^D$ 
# -----

def lam_symbol_Td(L, D, device):
    """
     $\lambda(k) = \sum_i (2 - 2 \cos k_i)$ , with  $k_i = 2\pi n_i / L$ .
    Returns  $\lambda$  array shape  $[L]^D$ .
    """
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k) # [L]
    # broadcast sum without meshgrid
    lam = t
    for _ in range(D-1):
        lam = lam.unsqueeze(-1)
    # lam is now [L,1,1,1] for D=4; add other axes terms
    for ax in range(1, D):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(D-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam # [L]^D

def torus_L1_dist_grid(L, D, device):
    """
     $\text{dist}[x] = \sum_i \min(x_i, L-x_i)$  on  $T^D$ . shape  $[L]^D$  (int32).
    """
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L) # [L]
```

```

    dist = dx
    for _ in range(D-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, D):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(D-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist # [L]*D

def env_by_dist(abs_field, dist, r_max):
    """
    env[r] = max_{dist==r} abs_field
    """
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtyp
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    eps = 1e-300
    y = torch.log(torch.clamp(env, min=eps))
    slopes = -(y[1:] - y[:-1]) # Δr=1
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
    return mids, slopes

def fit_stats(mids, slopes, rmin, rmax):
    mask = (mids >= rmin) & (mids <= rmax) & torch.isfinite(slopes)
    med = float(torch.median(slopes[mask]).item())
    mean = float(torch.mean(slopes[mask]).item())
    return med, mean, int(mask.sum().item())

# -----
# Exact C0 for K = (Δ)^R (movement-only row-sum), via real-space kernel
# -----

def laplacian_scalar(f):
    # f shape [L]*D on torus
    D = f.ndim
    out = torch.zeros_like(f)
    for ax in range(D):
        out = out + (2.0*f - torch.roll(f, +1, dims=ax) - torch.roll(f, -1,
    return out

def kernel_row_C0(L, D, R_power):
    """
    Computes C0 = ∑_{|x|=1}^D ∑_{|y|=1}^D |K(x,y)| for K=(Δ)^R power on scalar fields

```

```

    Computes  $G_0 = \sum_{k=1}^R \frac{1}{k^2} \nabla^k \phi(\Delta)$  for  $R \in \mathbb{N}$ ,  $R\_power$  on scalar fields.
    (This matches the operator used in your previous matrix-free runs for 1-
    """

    f = torch.zeros([L]*D, device=device)
    f[(0,)*D] = 1.0
    g = f
    for _ in range(R_power):
        g = laplacian_scalar(g)
    g_abs = torch.abs(g)
    g_abs[(0,)*D] = 0.0
    return float(g_abs.sum().item())

# -----
# FFT Green's function for  $M = m^2 I + \alpha (\Delta)^R$ 
# Remove  $k=0$  mode to mimic your "project out constant 1-forms" step.
# -----

def greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=True):
    lam = lam_symbol_Td(L, D, device=device)          # [L]*D
    denom = m2 + alpha * (lam ** R_power)             # [L]*D
    if remove_k0:
        denom[(0,)*D] = float("inf")                 # zero out constant
    Gk = 1.0 / denom
    g = torch.fft.ifftn(Gk).real                      # [L]*D, real symme
    return g

# -----
# Directional probes (to explain why env-slope < axis  $\kappa$ )
# -----

def directional_profile(g, direction, L, D):
    """
    Sample  $|g(r*\text{dir})|$  for  $r=0..L//2$  along integer direction vector `direction`
    direction entries must be ints.
    """
    direction = list(direction)
    assert len(direction) == D
    rmax = L//2
    vals = torch.zeros(rmax+1, device=g.device, dtype=torch.float64)
    for r in range(rmax+1):
        idx = tuple((r*direction[i]) % L for i in range(D))
        vals[r] = torch.abs(g[idx])
    return vals

def directional_slope(vals, rmin, rmax):
    eps = 1e-300
    y = torch.log(torch.clamp(vals, min=eps))
    dy = y[1:] - y[:-1]
    slopes = -dy
    mids = torch.arange(0.5, 0.5+slopes.numel(), device=vals.device, dtype=t
    mask = (mids >= rmin) & (mids <= rmax)

```

```

        return float(torch.median(slopes[mask]).item()), float(torch.mean(slopes

# -----
# Run suite (matches your previous outputs, but exact via FFT)
# -----

D = 4
L = 32
m2 = 0.3
alpha = 1.0

# fit window (vertex L1 metric) - same as your previous
rmin_fit = 6.0
rmax_fit = 18.0

dist = torus_L1_dist_grid(L, D, device=device)
r_max_total = D * (L//2)

print(f"device={device}")
print()

for R_power in [1, 2, 3]:
    g = greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=True)
    abs_g = torch.abs(g)

    env = env_by_dist(abs_g, dist, r_max_total)
    mids, slopes = local_slopes(env)
    c_med, c_mean, npts = fit_stats(mids, slopes, rmin_fit, rmax_fit)

    # exact ledger constants for  $K=(\Delta)^R$ 
    C0 = kernel_row_C0(L=64, D=D, R_power=R_power) # use bigger L for kerne
    R_move = R_power
    eta_CT = (1.0/R_move) * math.log(1.0 + m2 / (2.0 * alpha * C0))

    # reference  $\kappa$  for R=1 along axis (exact imaginary-dispersion root)
    if R_power == 1:
        kappa_axis = math.acosh(1.0 + m2/(2.0*alpha)) # = 2 asinh(m/(2 sqrt
    else:
        kappa_axis = float("nan")

    print("=== FFT decay experiment (exact on torus) ===")
    print(f"d={D}, L={L}, Operator:  $M = m^2 I + \alpha (\Delta)^R$ , R={R_power}")
    print(f"m^2={m2}, alpha={alpha}, m={math.sqrt(m2):.6g}")
    print(f"Fit window mids in [{rmin_fit}, {rmax_fit}] (vertex L1 metric),
    print(f"c_local median = {c_med:.6g}")
    print(f"c_local mean   = {c_mean:.6g}")
    if R_power == 1:
        print(f"kappa_axis (exact) = acosh(1+m^2/(2alpha)) = {kappa_axis:.6g
    print(f"C0 (exact movement row-sum for K) = {C0:.6g} (computed from K0
    print(f"eta_CT (ledger) = (1/R) log(1 + m^2/(2  $\alpha$  C0)) = {eta_CT:.6g}")
    print()
```

```

# directional diagnostics (why env-slope differs from axis  $\kappa$ )
# directions: axis, 2-axis diagonal, 4-axis diagonal
axis = (1,0,0,0)
diag2 = (1,1,0,0)
diag4 = (1,1,1,1)

v_axis = directional_profile(g, axis, L, D)
v_d2 = directional_profile(g, diag2, L, D)
v_d4 = directional_profile(g, diag4, L, D)

s_axis_med, s_axis_mean = directional_slope(v_axis, rmin_fit, min(rmax_f
s_d2_med, s_d2_mean = directional_slope(v_d2, rmin_fit, min(rmax_f
s_d4_med, s_d4_mean = directional_slope(v_d4, rmin_fit, min(rmax_f

print("directional median slopes over same window (r is steps along that
print(f" axis (1,0,0,0): {s_axis_med:.6g}")
print(f" diag2 (1,1,0,0): {s_d2_med:.6g}")
print(f" diag4 (1,1,1,1): {s_d4_med:.6g}")
print()

# print small env table
for r in range(0, 25):
    val = float(env[r].item())
    logv = math.log(max(val, 1e-300))
    print(f"r={r:2d} env={val:.3e} log={logv:+.3f}")
print()

```

device=cuda

```

=== FFT decay experiment (exact on torus) ===
d=4, L=32, Operator:  $M = m^2 I + \alpha (\Delta)^R$ ,  $R=1$ 
 $m^2=0.3$ ,  $\alpha=1.0$ ,  $m=0.547723$ 
Fit window mids in [6.0, 18.0] (vertex L1 metric), bins=12
c_local median = 0.440021
c_local mean = 0.463479
 $\kappa_{\text{axis}}$  (exact) =  $\text{acosh}(1+m^2/(2\alpha)) = 0.541097$ 
 $C_0$  (exact movement row-sum for  $K$ ) = 8 (computed from  $K\delta$ )
 $\eta_{\text{CT}}$  (ledger) =  $(1/R) \log(1 + m^2/(2 \alpha C_0)) = 0.0185764$ 

directional median slopes over same window (r is steps along that direction):
axis (1,0,0,0): -0.0348479
diag2 (1,1,0,0): -0.0155002
diag4 (1,1,1,1): -0.00104356

r= 0 env=1.439e-01 log=-1.939
r= 1 env=2.426e-02 log=-3.719
r= 2 env=8.720e-03 log=-4.742
r= 3 env=4.529e-03 log=-5.397
r= 4 env=2.856e-03 log=-5.858
r= 5 env=1.396e-03 log=-6.574

```

```

r= 6  env=7.944e-04  log=-7.138
r= 7  env=5.000e-04  log=-7.601
r= 8  env=3.374e-04  log=-7.994
r= 9  env=2.003e-04  log=-8.516
r=10  env=1.269e-04  log=-8.972
r=11  env=8.437e-05  log=-9.380
r=12  env=5.810e-05  log=-9.753
r=13  env=3.642e-05  log=-10.220
r=14  env=2.346e-05  log=-10.660
r=15  env=1.532e-05  log=-11.086
r=16  env=1.002e-05  log=-11.511
r=17  env=5.707e-06  log=-12.074
r=18  env=3.052e-06  log=-12.700
r=19  env=3.065e-06  log=-12.695
r=20  env=3.082e-06  log=-12.690
r=21  env=3.102e-06  log=-12.684
r=22  env=3.119e-06  log=-12.678
r=23  env=3.134e-06  log=-12.673
r=24  env=3.146e-06  log=-12.669

```

```

=== FFT decay experiment (exact on torus) ===
d=4, L=32, Operator:  $M = m^2 I + \alpha (\Delta)^R$ ,  $R=2$ 
 $m^2=0.3$ ,  $\alpha=1.0$ ,  $m=0.547723$ 
Fit window mids in [6.0, 18.0] (vertex L1 metric), bins=12
c_local median = 0.232472
c_local mean   = 0.37847
 $C0$  (exact movement row-sum for  $K$ ) = 184    (computed from  $K\delta$ )
 $\eta_{CT}$  (ledger) =  $(1/R) \log(1 + m^2/(2 \alpha C0)) = 0.000407443$ 

```

```

directional median slopes over same window (r is steps along that direction):
axis (1,0,0,0):      0.286214
diag2 (1,1,0,0):    -4.48346e-05
diag4 (1,1,1,1):    -6.02369e-05

```

```

r= 0  env=3.338e-02  log=-3.400

```

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# -----
# Symbols and Green's function on  $T^D$ 
# -----

def lam_symbol_Td(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k) # [L]
    lam = t
    for _ in range(D-1):
        lam = lam.unsqueeze(-1)
    for _ in range(1, D):

```

```

    for ax in range(1, D):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(D-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=False):
    lam = lam_symbol_Td(L, D, device=device)
    denom = m2 + alpha * (lam ** R_power)
    if remove_k0:
        denom[(0,)*D] = float("inf")
    Gk = 1.0 / denom
    g = torch.fft.ifftn(Gk).real
    return g

def torus_L1_dist_grid(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(D-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, D):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(D-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=slopes.dtype)
    return mids, slopes

def fit_stats(mids, slopes, rmin, rmax):
    mask = (mids >= rmin) & (mids <= rmax) & torch.isfinite(slopes)
    return (
        float(torch.median(slopes[mask]).item()),

```

```

        float(torch.mean(slopes[mask]).item()),
        int(mask.sum().item())
    )

# -----
# Directional decay diagnostics (fixed)
# -----

def directional_profile_abs(g, direction, L, D, rmax):
    direction = tuple(int(x) for x in direction)
    vals = torch.zeros(rmax+1, device=g.device, dtype=torch.float64)
    for r in range(rmax+1):
        idx = tuple((r*direction[i]) % L for i in range(D))
        vals[r] = torch.abs(g[idx])
    return vals

def tail_envelope(vals):
    # tail_env[r] = max_{s>=r} vals[s], monotone decreasing
    out = vals.clone()
    for r in range(out.numel()-2, -1, -1):
        out[r] = torch.maximum(out[r], out[r+1])
    return out

def directional_slope_from_tail(vals, rmin, rmax):
    tail = tail_envelope(vals)
    y = torch.log(torch.clamp(tail, min=1e-300))
    slopes = -(y[1:] - y[:-1]) # Δr=1
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=vals.device, dtype
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.median(slopes[mask]).item()), float(torch.mean(slopes

# -----
# Run (d=4) for R=1 only (tighten the manuscript narrative)
# -----

D = 4
L = 64          # bigger L => more safe mid-range; A100 can handle 64^4
m2 = 0.3
alpha = 1.0
R_power = 1

# Safe windows: keep well below wrap. L/2 = 32.
rmin_fit = 6.0
rmax_fit_shell = 20.0
rmax_dir = 24    # still < 32, avoids wrap artifacts along a line

g = greens_fft_Td(L, D, m2, alpha, R_power, remove_k0=False) # no need to d
abs_g = torch.abs(g)

# L1-shell envelope
dist = torus_l1dist_grid(L, D, device=device)
```

```

dist = torus_11_dist_grad(L, D, device=device,
r_max_total = D * (L//2)
env = env_by_dist(abs_g, dist, r_max_total)
mids, slopes = local_slopes(env)
c_med, c_mean, npts = fit_stats(mids, slopes, rmin_fit, rmax_fit_shell)

# Directional tail-envelope slopes (monotone by construction)
axis = (1,0,0,0)
diag2 = (1,1,0,0)
diag4 = (1,1,1,1)

v_axis = directional_profile_abs(g, axis, L, D, rmax_dir)
v_d2 = directional_profile_abs(g, diag2, L, D, rmax_dir)
v_d4 = directional_profile_abs(g, diag4, L, D, rmax_dir)

s_axis_med, s_axis_mean = directional_slope_from_tail(v_axis, rmin_fit, rmax_fit_shell)
s_d2_med, s_d2_mean = directional_slope_from_tail(v_d2, rmin_fit, rmax_fit_shell)
s_d4_med, s_d4_mean = directional_slope_from_tail(v_d4, rmin_fit, rmax_fit_shell)

kappa_axis = math.acosh(1.0 + m2/(2.0*alpha)) # exact axis benchmark on Z (

print("=== FFT (exact) decay diagnostics for  $M = m^2 I + \alpha \Delta$  on  $T^4$  ===")
print(f"device={device}, d={D}, L={L}, m^2={m2}, alpha={alpha}, m={math.sqrt(m2)}")
print(f"L1-shell fit mids in [{rmin_fit}, {rmax_fit_shell}] (bins={npts})")
print(f" c_shell median = {c_med:.6g}")
print(f" c_shell mean = {c_mean:.6g}")
print()
print(f"Directional (tail-envelope) median slopes, r in [{rmin_fit}, {rmax_fit_shell}]")
print(f" axis (1,0,0,0): {s_axis_med:.6g} (mean {s_axis_mean:.6g})")
print(f" diag2 (1,1,0,0): {s_d2_med:.6g} (mean {s_d2_mean:.6g})")
print(f" diag4 (1,1,1,1): {s_d4_med:.6g} (mean {s_d4_mean:.6g})")
print()
print(f"kappa_axis benchmark = acosh(1 + m^2/(2alpha)) = {kappa_axis:.6g}")
print()

# Print a short shell table
for r in range(0, 26):
    val = float(env[r].item())
    logv = math.log(max(val, 1e-300))
    print(f"r={r:2d} env={val:.3e} log={logv:+.3f}")

```

```

=== FFT (exact) decay diagnostics for  $M = m^2 I + \alpha \Delta$  on  $T^4$  ===
device=cuda, d=4, L=64, m^2=0.3, alpha=1.0, m=0.547723
L1-shell fit mids in [6.0, 20.0] (bins=14)
 c_shell median = 0.390352
 c_shell mean = 0.395472

Directional (tail-envelope) median slopes, r in [6.0, 23]
 axis (1,0,0,0): 0.64957 (mean 0.668088)
 diag2 (1,1,0,0): 0.875545 (mean 0.891714)
 diag4 (1,1,1,1): 1.19668 (mean 1.21183)

```

```
kappa_axis benchmark = acosh(1 + m^2/(2alpha)) = 0.541097
```

```
r= 0  env=1.439e-01  log=-1.939
r= 1  env=2.426e-02  log=-3.719
r= 2  env=8.724e-03  log=-4.742
r= 3  env=4.533e-03  log=-5.396
r= 4  env=2.859e-03  log=-5.857
r= 5  env=1.400e-03  log=-6.572
r= 6  env=7.976e-04  log=-7.134
r= 7  env=5.032e-04  log=-7.594
r= 8  env=3.406e-04  log=-7.985
r= 9  env=2.035e-04  log=-8.500
r=10  env=1.301e-04  log=-8.947
r=11  env=8.754e-05  log=-9.343
r=12  env=6.127e-05  log=-9.700
r=13  env=3.960e-05  log=-10.137
r=14  env=2.664e-05  log=-10.533
r=15  env=1.850e-05  log=-10.898
r=16  env=1.320e-05  log=-11.235
r=17  env=8.886e-06  log=-11.631
r=18  env=6.147e-06  log=-11.999
r=19  env=4.352e-06  log=-12.345
r=20  env=3.142e-06  log=-12.671
r=21  env=2.168e-06  log=-13.041
r=22  env=1.527e-06  log=-13.392
r=23  env=1.095e-06  log=-13.725
r=24  env=7.971e-07  log=-14.042
r=25  env=5.592e-07  log=-14.397
```

```
import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

def lam_symbol_Td(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k)
    lam = t
    for _ in range(D-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, D):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(D-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, D, m2, alpha):
```

```

def greens_frc_fr(L, D, m2, alpha):
    lam = lam_symbol_Td(L, D, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    g = torch.fft.ifftn(Gk).real
    return g

def torus_L1_dist_grid(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(D-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, D):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(D-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
    return mids, slopes

def fit_med(mids, slopes, rmin, rmax):
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.median(slopes[mask]).item())

def directional_profile_abs(g, direction, L, D, rmax):
    direction = tuple(int(x) for x in direction)
    vals = torch.zeros(rmax+1, device=g.device, dtype=torch.float64)
    for r in range(rmax+1):
        idx = tuple((r*direction[i]) % L for i in range(D))
        vals[r] = torch.abs(g[idx])
    return vals

def tail_envelope(vals):
    out = vals.clone()
    for r in range(out.numel()-2, -1, -1):
        out[r] = torch.maximum(out[r], out[r+1])

```

```

    return out

def dir_rate_per_step(g, direction, L, D, rmin, rmax):
    vals = directional_profile_abs(g, direction, L, D, rmax)
    tail = tail_envelope(vals)
    mids, slopes = local_slopes(tail)
    return fit_med(mids, slopes, rmin, rmax-1)

# ---- parameters ----
D = 4
L = 64
alpha = 1.0
rmin = 6.0
rmax_shell = 20.0
rmax_dir = 24

dist = torus_L1_dist_grid(L, D, device=device)
r_max_total = D*(L//2)

axis = (1,0,0,0)
diag2 = (1,1,0,0)
diag4 = (1,1,1,1)

def l1_step(direction):
    return sum(abs(int(x)) for x in direction)

print(f"device={device}, d={D}, L={L}, alpha={alpha}")
print("m2      m      c_shell   axis/L1   diag2/L1   diag4/L1   m/sqrt(d)")
for m2 in [0.05, 0.1, 0.2, 0.3, 0.5, 1.0]:
    g = greens_fft_Td(L, D, m2, alpha)
    abs_g = torch.abs(g)

    env = env_by_dist(abs_g, dist, r_max_total)
    mids, slopes = local_slopes(env)
    c_shell = fit_med(mids, slopes, rmin, rmax_shell)

    s_axis = dir_rate_per_step(g, axis, L, D, rmin, rmax_dir) / l1_step(a
    s_d2 = dir_rate_per_step(g, diag2, L, D, rmin, rmax_dir) / l1_step(d
    s_d4 = dir_rate_per_step(g, diag4, L, D, rmin, rmax_dir) / l1_step(d

    m = math.sqrt(m2)
    msd = m / math.sqrt(D)

    print(f"{m2:<6.2g} {m:<7.4f} {c_shell:<9.5f} {s_axis:<9.5f} {s_d2:<9.5f}

```

```

device=cuda, d=4, L=64, alpha=1.0
m2      m      c_shell   axis/L1   diag2/L1   diag4/L1   m/sqrt(d)
0.05    0.2236  0.22713   0.33393   0.21190   0.13784   0.11180
0.1     0.3162  0.27489   0.42448   0.27663   0.18433   0.15811
0.2     0.4472  0.34319   0.55230   0.36798   0.24939   0.22361

```

0.3	0.5477	0.39035	0.64957	0.43777	0.29917	0.27386
0.5	0.7071	0.45451	0.80155	0.54760	0.37779	0.35355
1	1.0000	0.59238	1.07131	0.74568	0.52109	0.50000

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# ---- core FFT Green + envelopes (same as before) ----
def lam_symbol_Td(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k)
    lam = t
    for _ in range(D-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, D):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(D-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, D, m2, alpha):
    lam = lam_symbol_Td(L, D, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    g = torch.fft.ifftn(Gk).real
    return g

def torus_L1_dist_grid(L, D, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(D-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, D):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(D-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtyp

```

```

    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
    return mids, slopes

def fit_med(mids, slopes, rmin, rmax):
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.median(slopes[mask]).item())

# ---- run table ----
D = 4
L = 64
alpha = 1.0
rmin = 6.0
rmax_shell = 20.0

dist = torus_L1_dist_grid(L, D, device=device)
r_max_total = D*(L//2)

print(f"device={device}, d={D}, L={L}, alpha={alpha}")
print("m2      c_shell    eta_CT      ratio c_shell/eta_CT    log10(ratio)")
for m2 in [0.05, 0.1, 0.2, 0.3, 0.5, 1.0]:
    g = greens_fft_Td(L, D, m2, alpha)
    abs_g = torch.abs(g)

    env = env_by_dist(abs_g, dist, r_max_total)
    mids, slopes = local_slopes(env)
    c_shell = fit_med(mids, slopes, rmin, rmax_shell)

    eta_CT = math.log(1.0 + m2/(16.0*alpha)) # R=1, C0=8 for d=4 Laplacian-

    ratio = c_shell / eta_CT
    log10r = math.log10(ratio)

    print(f"{m2:<6.2g}  {c_shell:<9.5f} {eta_CT:<10.6f} {ratio:<20.3f} {log1
```

```

device=cuda, d=4, L=64, alpha=1.0
m2      c_shell    eta_CT      ratio c_shell/eta_CT    log10(ratio)
0.05    0.22713    0.003120    72.797    +1.862
0.1     0.27489    0.006231    44.120    +1.645
0.2     0.34319    0.012423    27.627    +1.441
0.3     0.39035    0.018576    21.013    +1.322
0.5     0.45451    0.030772    14.770    +1.169
1       0.59238    0.060675    9.771     +0.990
```

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# -----
# FFT Green's function for  $M = m^2 I + \alpha \Delta$  on  $T^d$ 
# -----

def lam_symbol_Td(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k) # [L]
    lam = t
    for _ in range(d-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, d):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(d-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, d, m2, alpha):
    lam = lam_symbol_Td(L, d, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    g = torch.fft.ifftn(Gk).real
    return g

def torus_L1_dist_grid(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(d-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, d):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(d-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)

```

```

env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.type)
for r in range(r_max+1):
    m = (dist == r)
    if torch.any(m):
        env[r] = torch.max(abs_field[m])
return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
return mids, slopes

# -----
# Predictive exponent E(x) via 1D root-find for lambda
# E(x) = sum x_i asinh(x_i/lambda) subject to sum(sqrt(1+(x_i/lambda)^2)-1)=
# -----

def solve_lambda_for_x(x, C):
    # x: list of ints length d, nonnegative, not all zero
    # Find lambda > 0 s.t. F(lambda)=0 where F(lambda)=sum(sqrt(1+(x_i/l)^2)
    # Monotone decreasing in lambda.
    xs = [float(v) for v in x]
    if sum(xs) == 0.0:
        return float("inf")

    def F(lam):
        s = 0.0
        for xi in xs:
            if xi == 0.0:
                continue
            t = xi / lam
            s += math.sqrt(1.0 + t*t) - 1.0
        return s - C

    # Bracket: lam_hi big => F<0; lam_lo small => F>0
    lam_hi = 1.0
    while F(lam_hi) > 0.0:
        lam_hi *= 2.0
        if lam_hi > 1e12:
            break

    lam_lo = lam_hi
    while F(lam_lo) < 0.0:
        lam_lo *= 0.5
        if lam_lo < 1e-18:
            break

    # Bisection
    for _ in range(80):
        lam_mid = 0.5 * (lam_lo + lam_hi)

```

```

        if F(lam_mid) > 0.0:
            lam_lo = lam_mid
        else:
            lam_hi = lam_mid
    return 0.5 * (lam_lo + lam_hi)

def E_of_x(x, C):
    lam = solve_lambda_for_x(x, C)
    s = 0.0
    for xi in x:
        if xi == 0:
            continue
        s += xi * math.asinh(xi / lam)
    return s

def compositions_4(r):
    # all nonnegative integer 4-tuples summing to r
    out = []
    for a in range(r+1):
        for b in range(r-a+1):
            for c in range(r-a-b+1):
                d = r - a - b - c
                out.append((a,b,c,d))
    return out

def predict_shell_Emin(d, r_max, m2, alpha):
    assert d == 4, "this quick enumerator is coded for d=4"
    C = m2 / (2.0 * alpha)
    Emin = [0.0] * (r_max+1)
    argmin = [(0,0,0,0)] * (r_max+1)
    for r in range(1, r_max+1):
        bestE = float("inf")
        bestx = None
        for x in compositions_4(r):
            # sort descending to avoid duplicates? not needed; count is small
            E = E_of_x(x, C)
            if E < bestE:
                bestE = E
                bestx = x
        Emin[r] = bestE
        argmin[r] = bestx
    return Emin, argmin

# -----
# Run comparison
# -----

d = 4
L = 64
m2 = 0.3
alpha = 1.0

```

```

# match your previous shell-fit window
rmin_fit = 6
rmax_fit = 20

print(f"device={device}, d={d}, L={L}, m2={m2}, alpha={alpha}")

# Measured env via FFT
g = greens_fft_Td(L, d, m2, alpha)
abs_g = torch.abs(g)
dist = torus_L1_dist_grid(L, d, device=device)
r_max_total = d * (L//2)
env = env_by_dist(abs_g, dist, r_max_total)

# Measured slopes
mids, slopes = local_slopes(env)

# Predict Emin(r) up to rmax_fit+1
Emin, argmin = predict_shell_Emin(d=d, r_max=rmax_fit+1, m2=m2, alpha=alpha)
pred_slope = [None] * (rmax_fit+1)
for r in range(1, rmax_fit+1):
    pred_slope[r] = Emin[r] - Emin[r-1]

# Print table on fit window
print("\nr    log(env[r])    meas_slope(mid=r-0.5)    pred_Emin[r]    pred_sl
for r in range(rmin_fit, rmax_fit+1):
    logenv = float(torch.log(torch.clamp(env[r], min=1e-300)).item())
    # slope at midpoint r-0.5 is slopes[r-1]
    ms = float(slopes[r-1].item())
    print(f"{r:2d}    {logenv:+12.6f}    {ms:10.6f}    {Emin[r]:10.6f}

# Summary: median slope compare
mask = (mids >= rmin_fit) & (mids <= rmax_fit)
c_meas_med = float(torch.median(slopes[mask]).item())
c_meas_mean = float(torch.mean(slopes[mask]).item())

pred_mid_slopes = [pred_slope[r] for r in range(rmin_fit, rmax_fit+1)]
pred_med = float(torch.tensor(pred_mid_slopes, dtype=torch.float64).median())
pred_mean = float(torch.tensor(pred_mid_slopes, dtype=torch.float64).mean())

print("\nSummary on window:")
print(f"measured median slope = {c_meas_med:.6g}, mean = {c_meas_mean:.6g}")
print(f"predicted median slope = {pred_med:.6g}, mean = {pred_mean:.6g}")

```

```
device=cuda, d=4, L=64, m2=0.3, alpha=1.0
```

r	log(env[r])	meas_slope(mid=r-0.5)	pred_Emin[r]	pred_slope[r]	a
6	-7.133916	0.562337	1.724800	0.282527	(1, 2,
7	-7.594478	0.460562	1.967762	0.242962	(1, 2,
8	-7.984830	0.390352	2.184101	0.216339	(2, 2,

9	-8.499832	0.515002	2.500929	0.316828	(2, 2,
10	-8.947139	0.447307	2.782949	0.282020	(2, 2,
11	-9.343367	0.396228	3.039418	0.256470	(2, 3,
12	-9.700147	0.356780	3.276151	0.236733	(3, 3,
13	-10.136671	0.436524	3.579673	0.303522	(3, 4,
14	-10.533278	0.396607	3.860067	0.280394	(3, 4,
15	-10.897592	0.364314	4.121834	0.261767	(3, 4,
16	-11.235348	0.337756	4.368202	0.246368	(4, 4,
17	-11.631059	0.395710	4.664603	0.296401	(4, 4,
18	-11.999494	0.368435	4.943750	0.279147	(4, 5,
19	-12.344869	0.345375	5.208285	0.264536	(5, 5,
20	-12.670517	0.325648	5.460252	0.251967	(5, 5,

Summary on window:

measured median slope = 0.390352, mean = 0.395472

predicted median slope = 0.264536, mean = 0.267865

```
import math

def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def predict_c_shell(m2, alpha=1.0, d=4, rstar=13.0):
    # choose k that dominates max envelope at radius rstar
    best = None
    for k in range(1, d+1):
        tk = t_k(m2, alpha, k)
        Phi = tk*rstar + 0.5*(k-1)*math.log(rstar)
        cand = (Phi, k, tk)
        if best is None or cand < best:
            best = cand
    Phi, kstar, c_pred = best
    return kstar, c_pred, Phi

# your measured c_shell from the FFT table (d=4, L=64, alpha=1), fit window
measured = {
    0.05: 0.22713,
    0.1: 0.27489,
    0.2: 0.34319,
    0.3: 0.39035,
    0.5: 0.45451,
    1.0: 0.59238,
}

alpha = 1.0
d = 4
rstar = 13.0 # midpoint-ish of your fit window

print(f"Predictive model for L1-shell max envelope (using r*={rstar})")
print("m2      k*    c_pred    c_meas    abs_err")
for m2, c_meas in measured.items():
```

```
kstar, c_pred, _ = predict_c_shell(m2, alpha=alpha, d=d, rstar=rstar)
print(f"{m2:<6g}  {kstar:<2d}  {c_pred:<10.5f} {c_meas:<10.5f} {abs(c_pr
```

Predictive model for L1-shell max envelope (using $r^*=13.0$)

m2	k*	c_pred	c_meas	abs_err
0.05	1	0.22314	0.22713	0.00399
0.1	1	0.31492	0.27489	0.04003
0.2	2	0.31492	0.34319	0.02827
0.3	2	0.38492	0.39035	0.00543
0.5	2	0.49493	0.45451	0.04042
1	3	0.56962	0.59238	0.02276

```
import math
import numpy as np

# Your measured shell medians (from FFT, d=4, L=64, alpha=1, window r=6..20)
measured = {
    0.05: 0.22713,
    0.1: 0.27489,
    0.2: 0.34319,
    0.3: 0.39035,
    0.5: 0.45451,
    1.0: 0.59238,
}

d = 4
alpha = 1.0
rmin, rmax = 6, 20

def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def comb(n, k):
    if k < 0 or k > n: return 0
    return math.comb(n, k)

def log_Mk(r, d, k):
    # M_k(r) ~ choose k coords * choose signs * compositions of r into k pos
    # = C(d,k) * 2^k * C(r-1, k-1)
    if r <= 0: return -1e300
    if k == 0: return -1e300
    if r < k: return -1e300
    Mk = comb(d,k) * (2**k) * comb(r-1, k-1)
    return math.log(Mk)

def predict_c_shell(m2, a, b):
    """
    For each r in window, choose k minimizing:
    Phi_k(r) = t_k r + (a/2)(k-1) log r - b log M_k(r)
    Predict median slope as median_{r in window} t_{k(r)}.
    """
```

```

"""
ks = []
ts = []
for r in range(rmin, rmax+1):
    best = None
    for k in range(1, d+1):
        tk = t_k(m2, alpha, k)
        Phi = tk*r + 0.5*a*(k-1)*math.log(r) - b*log_Mk(r, d, k)
        cand = (Phi, k, tk)
        if best is None or cand < best:
            best = cand
    _, kstar, tkstar = best
    ks.append(kstar)
    ts.append(tkstar)
# predicted "median slope" over the window
ts_sorted = sorted(ts)
pred_med = ts_sorted[len(ts_sorted)//2]
return pred_med, ks

def rms_err(a, b):
    se = 0.0
    for m2, c_meas in measured.items():
        c_pred, _ = predict_c_shell(m2, a, b)
        se += (c_pred - c_meas)**2
    return math.sqrt(se/len(measured))

# Grid-search fit for (a,b). Small grid is fine.
a_grid = np.linspace(0.0, 2.0, 81) # prefactor weight
b_grid = np.linspace(0.0, 0.5, 101) # entropy weight

best = None
for a in a_grid:
    for b in b_grid:
        err = rms_err(float(a), float(b))
        cand = (err, float(a), float(b))
        if best is None or cand < best:
            best = cand

err_best, a_best, b_best = best
print(f"Best-fit parameters on your table: a={a_best:.3f}, b={b_best:.3f}, R
print()

print("m2      c_meas    c_pred    abs_err    k(r) summary (counts over r=6..
for m2, c_meas in measured.items():
    c_pred, ks = predict_c_shell(m2, a_best, b_best)
    counts = {k: ks.count(k) for k in range(1, d+1)}
    print(f"{m2:<6g}  {c_meas:<8.5f}  {c_pred:<8.5f}  {abs(c_pred-c_meas):<8.5

print("\nSanity: pure exponent only (a=0,b=0) vs best-fit")
for m2 in measured.keys():
    c0 = predict_c_shell(m2, 0.0, 0.0)

```

```

cb, _ = predict_c_shell(m2, 0.0, 0.0)
cb, _ = predict_c_shell(m2, a_best, b_best)
print(f"m2={m2:<4g}  pred(a=b=0)={c0:.5f}  pred(best)={cb:.5f}")

```

Best-fit parameters on your table: a=0.950, b=0.000, RMS err=0.02769

m2	c_meas	c_pred	abs_err	k(r) summary (counts over r=6..20)
0.05	0.22713	0.22314	0.00399	{1: 15, 2: 0, 3: 0, 4: 0}
0.1	0.27489	0.31492	0.04003	{1: 8, 2: 7, 3: 0, 4: 0}
0.2	0.34319	0.31492	0.02827	{1: 2, 2: 13, 3: 0, 4: 0}
0.3	0.39035	0.38492	0.00543	{1: 0, 2: 15, 3: 0, 4: 0}
0.5	0.45451	0.49493	0.04042	{1: 0, 2: 9, 3: 6, 4: 0}
1	0.59238	0.56962	0.02276	{1: 0, 2: 2, 3: 11, 4: 2}

Sanity: pure exponent only (a=0,b=0) vs best-fit

m2=0.05	pred(a=b=0)=0.11175	pred(best)=0.22314
m2=0.1	pred(a=b=0)=0.15795	pred(best)=0.31492
m2=0.2	pred(a=b=0)=0.22314	pred(best)=0.31492
m2=0.3	pred(a=b=0)=0.27301	pred(best)=0.38492
m2=0.5	pred(a=b=0)=0.35174	pred(best)=0.49493
m2=1	pred(a=b=0)=0.49493	pred(best)=0.56962

```

import math
import numpy as np

measured = {
    0.05: 0.22713,
    0.1: 0.27489,
    0.2: 0.34319,
    0.3: 0.39035,
    0.5: 0.45451,
    1.0: 0.59238,
}

d = 4
alpha = 1.0
rmin, rmax = 6, 20

def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def Phi_k(m2, alpha, k, r, a=1.0):
    return t_k(m2, alpha, k)*r + 0.5*a*(k-1)*math.log(r)

def choose_k_per_r(m2, alpha, r, a=1.0):
    best = None
    for k in range(1, d+1):
        val = Phi_k(m2, alpha, k, r, a=a)
        cand = (val, k)
        if best is None or cand < best:
            best = cand

```

```

    return best[1]

def predict_log_env_curve(m2, alpha, rmax, a=1.0):
    """
    Build a synthetic log envelope curve:
    log_env_pred(0)=0
    log_env_pred(r) = - sum_{s=1..r} t_{k(s)}(m2)
    This models env(r) ~ exp(-∫ t_{k(s)} ds) with crossover.
    """
    logenv = [0.0]*(rmax+1)
    for r in range(1, rmax+1):
        k = choose_k_per_r(m2, alpha, r, a=a)
        logenv[r] = logenv[r-1] - t_k(m2, alpha, k)
    return logenv

def median_slope_from_curve(logenv, rmin, rmax):
    slopes = []
    for r in range(rmin, rmax+1):
        slopes.append(-(logenv[r] - logenv[r-1]))
    slopes.sort()
    return slopes[len(slopes)//2], sum(slopes)/len(slopes)

print("Refined predictor with a=1.0 (no fitting), b=0")
print("m2      c_meas    c_pred_med  c_pred_mean  abs_err")
for m2, c_meas in measured.items():
    logenv = predict_log_env_curve(m2, alpha, rmax, a=1.0)
    c_med, c_mean = median_slope_from_curve(logenv, rmin, rmax)
    print(f"{m2:<6g}  {c_meas:<8.5f} {c_med:<10.5f} {c_mean:<11.5f} {abs(c_m
```

```

Refined predictor with a=1.0 (no fitting), b=0
m2      c_meas    c_pred_med  c_pred_mean  abs_err
0.05    0.22713  0.22314    0.22314     0.00399
0.1     0.27489  0.31492    0.27821     0.04003
0.2     0.34319  0.31492    0.34065     0.02827
0.3     0.39035  0.38492    0.38492     0.00543
0.5     0.45451  0.49493    0.46511     0.04042
1       0.59238  0.56962    0.59432     0.02276

```

```

import math
import numpy as np

measured = {
    0.05: 0.22713,
    0.1: 0.27489,
    0.2: 0.34319,
    0.3: 0.39035,
    0.5: 0.45451,
    1.0: 0.59238,
}
```

```

d = 4
alpha = 1.0
rmin, rmax = 6, 20

def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def choose_k_per_r(m2, alpha, r, a=1.0):
    best = None
    for k in range(1, d+1):
        Phi = t_k(m2, alpha, k)*r + 0.5*a*(k-1)*math.log(r)
        cand = (Phi, k)
        if best is None or cand < best:
            best = cand
    return best[1]

def predict_log_env_curve(m2, alpha, rmax, a=1.0):
    logenv = [0.0]*(rmax+1)
    for r in range(1, rmax+1):
        k = choose_k_per_r(m2, alpha, r, a=a)
        logenv[r] = logenv[r-1] - t_k(m2, alpha, k)
    return np.array(logenv, dtype=float)

def smooth(y, w=3):
    # centered moving average of width w (odd)
    assert w % 2 == 1
    h = w//2
    out = y.copy()
    for i in range(len(y)):
        lo = max(0, i-h)
        hi = min(len(y), i+h+1)
        out[i] = y[lo:hi].mean()
    return out

def slope_stats_from_logenv(logenv, rmin, rmax):
    slopes = []
    for r in range(rmin, rmax+1):
        slopes.append(-(logenv[r] - logenv[r-1]))
    slopes_sorted = sorted(slopes)
    med = slopes_sorted[len(slopes_sorted)//2]
    mean = sum(slopes)/len(slopes)
    return med, mean

print("Predict median slope via smoothed predicted log-env curve (a=1, b=0)")
print("m2      c_meas    pred_med(w=1)  pred_med(w=3)  pred_med(w=5)  pred_m
for m2, c_meas in measured.items():
    base = predict_log_env_curve(m2, alpha, rmax, a=1.0)
    med1, mean1 = slope_stats_from_logenv(base, rmin, rmax)
    sm3 = smooth(base, w=3)
    med3, mean3 = slope_stats_from_logenv(sm3, rmin, rmax)
    _
    ..
    _

```

```

sm5 = smooth(base, w=5)
med5, mean5 = slope_stats_from_logenv(sm5, rmin, rmax)
print(f"{m2:<6g}  {c_meas:<8.5f}  {med1:<12.5f}  {med3:<12.5f}  {med5:<12.5f}")

```

```

Predict median slope via smoothed predicted log-env curve (a=1, b=0)
m2      c_meas      pred_med(w=1)  pred_med(w=3)  pred_med(w=5)  pred_mean(w=3)
0.05    0.22713    0.22314      0.22314      0.22314      0.21571
0.1     0.27489    0.31492      0.31492      0.29657      0.27077
0.2     0.34319    0.31492      0.31492      0.31492      0.33016
0.3     0.39035    0.38492      0.38492      0.38492      0.37556
0.5     0.45451    0.49493      0.49493      0.49493      0.45159
1       0.59238    0.56962      0.56962      0.56962      0.57534

```

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# FFT env
def lam_symbol_Td(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k)
    lam = t
    for _ in range(d-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, d):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(d-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, d, m2, alpha):
    lam = lam_symbol_Td(L, d, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    return torch.fft.ifftn(Gk).real

def torus_L1_dist_grid(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(d-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, d):
        dd = dx
        for _ in range(ax):

```

```

        dd = dd.unsqueeze(0)
    for _ in range(d-ax-1):
        dd = dd.unsqueeze(-1)
    dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
    return mids, slopes

def mean_slope_over_window(env, rmin, rmax):
    mids, slopes = local_slopes(env)
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.mean(slopes[mask]).item())

# predictor (no knobs): choose k per r via exponent+prefactor
def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def choose_k(m2, alpha, d, r):
    best = None
    for k in range(1, d+1):
        Phi = t_k(m2, alpha, k)*r + 0.5*(k-1)*math.log(r) # a=1
        cand = (Phi, k)
        if best is None or cand < best:
            best = cand
    return best[1]

def pred_mean_slope(m2, alpha, d, rmin, rmax):
    ks = []
    ts = []
    for r in range(rmin, rmax+1):
        k = choose_k(m2, alpha, d, r)
        ks.append(k)
        ts.append(t_k(m2, alpha, k))
    counts = {k: ks.count(k) for k in range(1, d+1)}
    return sum(ts)/len(ts), counts

# run
d = 4
l = 64

```

```

L = 64
alpha = 1.0
rmin, rmax = 6, 20

dist = torus_L1_dist_grid(L, d, device=device)
r_max_total = d*(L//2)

print(f"device={device}, d={d}, L={L}, alpha={alpha}, window r={rmin}..{rmax}")
print("m2      meas_mean  pred_mean  abs_err   k-counts (over r window)")
for m2 in [0.05, 0.1, 0.2, 0.3, 0.5, 1.0]:
    g = greens_fft_Td(L, d, m2, alpha)
    env = env_by_dist(torch.abs(g), dist, r_max_total)
    meas = mean_slope_over_window(env, rmin, rmax)
    pred, counts = pred_mean_slope(m2, alpha, d, rmin, rmax)
    print(f"{m2:<6g}  {meas:<9.5f} {pred:<9.5f} {abs(meas-pred):<8.5f} {counts}")

```

```

device=cuda, d=4, L=64, alpha=1.0, window r=6..20
m2      meas_mean  pred_mean  abs_err   k-counts (over r window)
0.05    0.24463    0.22314    0.02149   {1: 15, 2: 0, 3: 0, 4: 0}
0.1     0.28687    0.27821    0.00866   {1: 9, 2: 6, 3: 0, 4: 0}
0.2     0.34802    0.34065    0.00737   {1: 3, 2: 12, 3: 0, 4: 0}
0.3     0.39547    0.38492    0.01055   {1: 0, 2: 15, 3: 0, 4: 0}
0.5     0.47108    0.46511    0.00597   {1: 0, 2: 10, 3: 5, 4: 0}
1       0.60994    0.59432    0.01562   {1: 0, 2: 3, 3: 12, 4: 0}

```

```

import math
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

def lam_symbol_Td(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k)
    lam = t
    for _ in range(d-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, d):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(d-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, d, m2, alpha):
    lam = lam_symbol_Td(L, d, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    . . .

```

```

        return torch.tanh(tn(Gk)).real

def torus_L1_dist_grid(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(d-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, d):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(d-ax-1):
            dd = dd.unsqueeze(-1)
        dist = dist + dd
    return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=
    return mids, slopes

def mean_slope_over_window(env, rmin, rmax):
    mids, slopes = local_slopes(env)
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.mean(slopes[mask]).item())

def t_k(m2, alpha, k):
    return math.acosh(1.0 + m2/(2.0*alpha*k))

def choose_k(m2, alpha, d, r):
    best = None
    for k in range(1, d+1):
        Phi = t_k(m2, alpha, k)*r + 0.5*(k-1)*math.log(r)
        cand = (Phi, k)
        if best is None or cand < best:
            best = cand
    return best[1]

def pred_mean_slope(m2, alpha, d, rmin, rmax):
    ts = []
    for r in range(rmin, rmax+1):

```

```

        k = choose_k(m2, alpha, d, r)
        ts.append(t_k(m2, alpha, k))
    return sum(ts)/len(ts)

# settings
d = 4
L = 96 # bigger so we can safely push window out; A100 should handle
alpha = 1.0
m2_list = [0.05, 0.1, 0.2, 0.3, 0.5, 1.0]
windows = [(6,20), (10,28), (12,36), (16,44)]

dist = torus_L1_dist_grid(L, d, device=device)
r_max_total = d*(L//2)

print(f"device={device}, d={d}, L={L}, alpha={alpha}")
for (rmin, rmax) in windows:
    print(f"\nWindow r={rmin}..{rmax}")
    print("m2      meas_mean  pred_mean  abs_err")
    for m2 in m2_list:
        g = greens_fft_Td(L, d, m2, alpha)
        env = env_by_dist(torch.abs(g), dist, r_max_total)
        meas = mean_slope_over_window(env, rmin, rmax)
        pred = pred_mean_slope(m2, alpha, d, rmin, rmax)
        print(f"{m2:<6g}  {meas:<9.5f}  {pred:<9.5f}  {abs(meas-pred):.5f}")

```

```
device=cuda, d=4, L=96, alpha=1.0
```

```
Window r=6..20
```

m2	meas_mean	pred_mean	abs_err
0.05	0.24463	0.22314	0.02149
0.1	0.28687	0.27821	0.00866
0.2	0.34802	0.34065	0.00737
0.3	0.39547	0.38492	0.01055
0.5	0.47108	0.46511	0.00597
1	0.60994	0.59432	0.01562

```
Window r=10..28
```

m2	meas_mean	pred_mean	abs_err
0.05	0.20131	0.20942	0.00811
0.1	0.24527	0.24730	0.00203
0.2	0.30826	0.31492	0.00666
0.3	0.35684	0.36281	0.00597
0.5	0.43392	0.43372	0.00020
1	0.57481	0.53817	0.03664

```
Window r=12..36
```

m2	meas_mean	pred_mean	abs_err
0.05	0.18507	0.19185	0.00678
0.1	0.22995	0.23416	0.00421
0.2	0.29399	0.29884	0.00485
0.3	0.34325	0.34572	0.00247
0.5	0.42126	0.41118	0.01008

```
1      0.56355  0.52182  0.04173
```

```
Window r=16..44
```

m2	meas_mean	pred_mean	abs_err
0.05	0.16911	0.17818	0.00907
0.1	0.21441	0.22314	0.00874
0.2	0.27883	0.28522	0.00639
0.3	0.32830	0.33182	0.00352
0.5	0.40657	0.38323	0.02333
1	0.54919	0.50781	0.04138

```
import math
import numpy as np
import torch

torch.set_default_dtype(torch.float64)
device = "cuda" if torch.cuda.is_available() else "cpu"

# --- FFT measured mean slope as before ---
def lam_symbol_Td(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.float64)
    k = 2.0 * math.pi * n / L
    t = 2.0 - 2.0 * torch.cos(k)
    lam = t
    for _ in range(d-1):
        lam = lam.unsqueeze(-1)
    for ax in range(1, d):
        tt = t
        for _ in range(ax):
            tt = tt.unsqueeze(0)
        for _ in range(d-ax-1):
            tt = tt.unsqueeze(-1)
        lam = lam + tt
    return lam

def greens_fft_Td(L, d, m2, alpha):
    lam = lam_symbol_Td(L, d, device=device)
    Gk = 1.0 / (m2 + alpha * lam)
    return torch.fft.ifftn(Gk).real

def torus_L1_dist_grid(L, d, device):
    n = torch.arange(L, device=device, dtype=torch.int32)
    dx = torch.minimum(n, (L - n) % L)
    dist = dx
    for _ in range(d-1):
        dist = dist.unsqueeze(-1)
    for ax in range(1, d):
        dd = dx
        for _ in range(ax):
            dd = dd.unsqueeze(0)
        for _ in range(d-ax-1):
```

```

        for _ in range(r_max+1):
            dd = dd.unsqueeze(-1)
            dist = dist + dd
        return dist

def env_by_dist(abs_field, dist, r_max):
    env = torch.zeros(r_max+1, device=abs_field.device, dtype=abs_field.dtype)
    for r in range(r_max+1):
        m = (dist == r)
        if torch.any(m):
            env[r] = torch.max(abs_field[m])
    return env

def local_slopes(env):
    y = torch.log(torch.clamp(env, min=1e-300))
    slopes = -(y[1:] - y[:-1])
    mids = torch.arange(0.5, 0.5 + slopes.numel(), device=env.device, dtype=torch.float)
    return mids, slopes

def mean_slope_over_window(env, rmin, rmax):
    mids, slopes = local_slopes(env)
    mask = (mids >= rmin) & (mids <= rmax)
    return float(torch.mean(slopes[mask]).item())

# --- Sharp symbol-based predictor for L1-shell ---
# For u on simplex, define F_u(gamma)=2*sum_i (cosh(gamma u_i)-1).
# For given gamma, the best (smallest) F_u is achieved by concentrating u (axis)
# and the worst by spreading u (diagonal). For L1-shell MAX envelope, the slow
# corresponds to MIN gamma such that constraint holds for SOME direction that
# Empirically the max is dominated by "spread" directions, so we take:
# c_pred = sup gamma s.t. m2 >= 2 alpha * min_{u in simplex} sum(cosh(gamma u_i)-1)
# and we compute the minimizer u by numerical optimization on the simplex.

def min_F_over_simplex(gamma, d=4, ngrid=6000, rng=0):
    # crude but deterministic: sample Dirichlet points + include corners and eq
    rs = np.random.RandomState(rng)
    best = float("inf")

    # include corners (axis) and equal split
    corners = [np.eye(d)[i] for i in range(d)]
    eq = np.ones(d)/d
    for u in corners + [eq]:
        s = sum(math.cosh(gamma*ui)-1.0 for ui in u)
        best = min(best, s)

    # sample Dirichlet(1,1,1,1) points
    for _ in range(ngrid):
        y = rs.exponential(1.0, size=d)
        u = y / y.sum()
        s = sum(math.cosh(gamma*ui)-1.0 for ui in u)
        if s < best:

```

```

        best = s
    return best

def c_pred_symbol(m2, alpha, d=4):
    # find largest gamma such that  $m2 \geq 2 \alpha * \min_u \sum(\cosh(\gamma u_i) - 1)$ 
    target = m2/(2.0*alpha)

    lo, hi = 0.0, 5.0
    # increase hi until infeasible
    while min_F_over_simplex(hi, d=d, ngrid=4000, rng=1) <= target:
        hi *= 1.5
        if hi > 50:
            break

    for _ in range(40):
        mid = 0.5*(lo+hi)
        val = min_F_over_simplex(mid, d=d, ngrid=4000, rng=1)
        if val <= target:
            lo = mid
        else:
            hi = mid
    return lo

# --- run comparison on your windows ---
d = 4
L = 96
alpha = 1.0
m2_list = [0.05, 0.1, 0.2, 0.3, 0.5, 1.0]
windows = [(6,20), (10,28), (12,36), (16,44)]

dist = torus_L1_dist_grid(L, d, device=device)
r_max_total = d*(L//2)

print(f"device={device}, d={d}, L={L}, alpha={alpha}")
for (rmin, rmax) in windows:
    print(f"\nWindow r={rmin}..{rmax}")
    print("m2      meas_mean  c_pred_symbol  abs_err")
    for m2 in m2_list:
        g = greens_fft_Td(L, d, m2, alpha)
        env = env_by_dist(torch.abs(g), dist, r_max_total)
        meas = mean_slope_over_window(env, rmin, rmax)
        pred = c_pred_symbol(m2, alpha, d=d)
        print(f"{m2:<6g}  {meas:<9.5f}  {pred:<12.5f}  {abs(meas-pred):.5f}")

```

```
device=cuda, d=4, L=96, alpha=1.0
```

```
Window r=6..20
```

m2	meas_mean	c_pred_symbol	abs_err
0.05	0.24463	0.44698	0.20235
0.1	0.28687	0.63180	0.34493
-	-	-	-

0.2	0.34802	0.89257	0.54455
0.3	0.39547	1.09205	0.69658
0.5	0.47108	1.40695	0.93587
1	0.60994	1.97973	1.36979

Window r=10..28

m2	meas_mean	c_pred_symbol	abs_err
0.05	0.20131	0.44698	0.24567
0.1	0.24527	0.63180	0.38653
0.2	0.30826	0.89257	0.58431
0.3	0.35684	1.09205	0.73521
0.5	0.43392	1.40695	0.97303
1	0.57481	1.97973	1.40492

Window r=12..36

m2	meas_mean	c_pred_symbol	abs_err
0.05	0.18507	0.44698	0.26191
0.1	0.22995	0.63180	0.40185
0.2	0.29399	0.89257	0.59858
0.3	0.34325	1.09205	0.74880
0.5	0.42126	1.40695	0.98569
1	0.56355	1.97973	1.41618

Window r=16..44

m2	meas_mean	c_pred_symbol	abs_err
0.05	0.16911	0.44698	0.27787
0.1	0.21441	0.63180	0.41739
0.2	0.27883	0.89257	0.61374
0.3	0.32830	1.09205	0.76375
0.5	0.40657	1.40695	1.00038
1	0.54919	1.97973	1.43054

